

华中科技大学

研究生（文献阅读与选题）报告

☐文献阅读、☐选题、☐文献阅读与选题、☐论文中期进展

题 目：大语言模型智能体故障诊断的证据链交互设计研究

学 号 M202570493

姓 名 陈思静

专 业 设计学

指 导 教 师 何心

院（系、所） 设计学院

填表注意事项

- 一、本表适用于攻读硕士、博士学位研究生文献阅读报告、选题报告、文献阅读与选题报告、论文中期进展报告。
- 二、以上各报告内容及要求由相关院（系、所）做具体要求。
- 三、以上各报告均须存入研究生个人学籍档案。
- 四、本表填写要求文句通顺、内容明确、字迹工整。
- 五、A4 纸正反打印

目 录

文献阅读卡	1
1 文献综述	42
1.1 大语言模型智能体故障诊断的研究基础	(42)
1.2 开发者故障分析的信息行为与认知基础	(44)
1.3 运行证据的组织与交互呈现	(47)
1.4 大语言模型智能体可观测与交互调试研究	(49)
1.5 证据呈现的设计研究与评价方法	(52)
1.6 国内外研究进展与本研究定位	(53)
1.7 本章小结	(54)
2 选题报告	56
2.1 研究背景	(56)
2.2 研究目的与意义	(57)
2.3 国内外研究现状	(57)
2.4 研究问题	(59)
2.5 核心概念与研究范围	(59)
2.6 研究内容与技术路线	(60)
2.7 研究材料与案例构建	(62)
2.8 评价指标与分析方法	(63)
2.9 研究概念框架	(64)
2.10 可行性、风险与伦理	(65)
2.11 进度安排	(66)
2.12 预期成果与创新点	(67)
参考文献	67

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：01

名称 Results-Actionability Gap: Understanding How Practitioners Evaluate LLM Products in the Wild

作者 Willem van der Maden; Malak Sadek; Ziang Xiao; Aske Mottelson; Q. Vera Liao; Jichen Zhu 译者

出处【美国】Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, 2026: 1-17 页

文献阅读心得（每篇不少于 150 字）：

研究背景 大语言模型（large language model, LLM）产品的评价对象不是孤立模型，而是包含提示词、检索、界面与业务流程的完整系统。传统基准分数很难直接回答产品团队下一步应改什么，实践者因此在不确定性、资源限制和多方目标之间组织评价工作。

研究问题 研究问题一（RQ1）：生产环境中的大语言模型产品目前采用哪些评价实践？

研究问题二（RQ2）：实践者认为评价大语言模型产品时最主要的困难是什么？

研究方法 作者对 19 名跨行业大语言模型产品实践者开展半结构化访谈，并进行主题分析。研究覆盖评价执行、评价方案设计及组织层面的协调工作，分析对象是面向真实用户的完整产品。

主要发现 研究归纳出 10 类评价活动和 5 类挑战。17 名参与者经历了“结果—可行动性鸿沟”：团队能收集反馈或分数，却难以判断失败来自提示、检索、模型还是其他部件。人工判断与“直觉检查”（vibe check）被视为对大语言模型特性的适应，而不只是方法不成熟。

贡献与意义 论文提出“规范化历程”（formalization journey）和“结果—可行动性鸿沟”（results-actionability gap），把评价后的定位、协商与改进纳入评价研究，并从较成熟团队中总结了可实施的组织与流程策略。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：02

名称 Interactive Debugging and Steering of Multi-Agent AI Systems

作者 Will Epperson; Gagan Bansal; Victor Dibia; Adam Fournery; Jack Gerrits; Erkang Zhu; Saleema Amershi 译者

出处【美国】Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, 2025: 156:1-156:15 页

文献阅读心得（每篇不少于 150 字）：

研究背景 多智能体（multi-agent）系统会产生数十甚至上百条消息，错误可能在早期计划、角色分工或工具调用中出现，并沿后续协作传播。现有开发工具多支持事后阅读日志，缺少暂停、回退和试改轨迹的交互能力。

研究问题 研究问题一（RQ1）：开发者在构建和调试多智能体工作流时面临哪些核心困难？

研究问题二（RQ2）：消息级浏览、编辑与重置如何支持故障诊断和智能体引导（agent steering）？

研究方法 研究先访谈 5 名智能体开发者，据此设计交互调试工具 AGDebugger；随后开展含 14 名参与者的两阶段用户研究。第一阶段比较 AGDebugger 与基线日志界面的错误识别，第二阶段让参与者编辑历史消息并重新运行。

主要发现 参与者通常花约 10 分钟阅读日志后才开始修改。5/6 的第一阶段参与者偏好 AGDebugger；第二阶段 8 人共做 24 次编辑，其中 14 次是补充具体指令，另有简化指令和修改计划。重置与编辑功能评分最高，但只有 2/8 得到完全正确答案。

贡献与意义 AGDebugger 把历史概览、逐条消息检查、回退、编辑与继续执行放进同一界面，使开发者能够用反事实式试验观察某次干预是否改变后续轨迹。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：03

名称 Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems

作者 Shaokun Zhang; Ming Yin; Jieyu Zhang; Jiale Liu; Zhiguang Han; Jingyang Zhang; Beibin Li; Chi Wang; Huazheng Wang; Yiran Chen; Qingyun Wu 译者

出处【美国】Proceedings of Machine Learning Research, 2025, 267: 76583-76599 页

文献阅读心得（每篇不少于 150 字）：

研究背景 多智能体任务失败时，最后输出错误的智能体不一定是决定结果的源头。有效调试需要同时定位责任智能体和关键步骤，但长轨迹、角色交互和错误传播使人工归因成本很高。

研究问题 研究问题一（RQ1）：能否自动识别导致任务失败的智能体，即回答“谁”造成失败？

研究问题二（RQ2）：能否定位该智能体产生决定性错误的具体步骤，即回答“何时”出错？

研究方法 作者将修正某个“智能体一时间”对后能否把失败变为成功作为决定性错误的反事实定义，并以最早的决定性错误为标注目标。Who&When 数据集包含 127 个多智能体系统、184 个失败归因任务，比较一次性分析（all-at-once）、二分搜索（binary search）和逐步分析（step-by-step）三类方法。

主要发现 最佳方法的智能体级准确率为 53.5%，步骤级准确率仅 14.2%，部分方法低于随机基线；包括先进推理模型在内的方法仍未达到实用水平。自动判断“谁”已有有限信号，而精确判断“何时”更困难。

贡献与意义 论文首次系统形式化多智能体自动故障归因，提供同时含责任智能体、决定性步骤和自然语言原因的细粒度数据集，并把智能体级与步骤级能力分开评价。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：04

名称 Finding AI's Faults with AAR/AI: An Empirical Study

作者 Roli Khanna; Jonathan Dodge; Andrew Anderson; Rupika Dikkala; Jed Irvine; Zeyad Shureih; Kin-Ho Lam; Caleb R. Matthews; Zhengxian Lin; Minsuk Kahng; Alan Fern; Margaret Burnett 译者

出处【美国】ACM Transactions on Interactive Intelligent Systems, 2022, 12(1): 1:1-1:33 页

文献阅读心得（每篇不少于 150 字）：

研究背景 总体成败或平均性能无法告诉领域用户人工智能究竟在哪些情境下不可靠。要决定何时依赖智能体，用户需要把问题定位到具体决策和内部预测，而不只是给出笼统的通过或失败判断。

研究问题 研究问题一（RQ1）：人工智能行动后复盘流程（After-Action Review for AI, AAR/AI）能否帮助具备领域知识的用户定位人工智能故障？

研究问题二（RQ2）：故障类型不同，用户发现故障的效果是否也不同？

研究方法 65 名有即时战略游戏经验但无人工智能或机器学习背景的参与者被随机分到复盘组和对照组，分析强化学习智能体完成《星际争霸 II》拉锯战任务的过程。研究用 10 个经验证的自然故障实例计算问题报告的召回率和精确率。

主要发现 AAR/AI 组召回率为 0.233，对照组为 0.080；精确率分别为 0.179 和 0.116，差异均显著。AAR/AI 参与者在每类故障上表现更好，发现任一特定故障的可能性平均接近对照组的 6 倍。

贡献与意义 论文用可核对的故障真值、召回率和精确率评价人类诊断，而非只测满意度；结构化复盘和标签活动还可能帮助用户从单个实例上升到故障类别。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：05

名称 Debugging Reinvented: Asking and Answering Why and Why Not Questions about Program Behavior

作者 Andrew J. Ko; Brad A. Myers 译者

出处 【美国】Proceedings of the 30th International Conference on Software Engineering, 2008: 301-310 页

文献阅读心得（每篇不少于 150 字）：

研究背景 开发者观察到异常输出后，传统调试器仍要求他们先猜测相关代码，再用断点和单步执行验证。由于初始猜测经常错误，问题表述与工具操作之间的转换会产生大量无关搜索。

研究问题 研究问题一（RQ1）：如何从程序代码和一次执行中自动提取用户可能提出的“为什么”与“为什么没有”（why and why-not）问题？

研究问题二（RQ2）：如何用静态、动态分析回答这些问题，并把答案映射到相关运行事件和代码？

研究方法 论文给出 Java Whyline 的系统设计与算法：记录完整执行历史，从文本和图形输出生成问题，结合动态切片、静态切片、调用图、可达性分析和值来源分析构造因果解释，并讨论性能与覆盖限制。

主要发现 “为什么发生”可沿已执行依赖回溯；“为什么没有发生”必须分析潜在定义、未满足条件与未到达路径，计算和问题空间更复杂。原型证明可以从可见输出进入相关代码，而无需先猜故障位置。

贡献与意义 文章把调试问题作为一等交互对象，并提出面向大型 Java 程序自动生成、筛选和回答“为什么”与“为什么没有”问题的完整技术架构。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：06

名称 Designing the Whyline: A Debugging Interface for Asking Questions about Program Behavior

作者 Andrew J. Ko; Brad A. Myers 译者

出处 【美国】 Proceedings of the 2004 ACM Conference on Human Factors in Computing Systems, 2004: 151-158 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试本质上从对运行行为的疑问开始，但断点、单步和打印语句迫使程序员先定位代码。Whyline 尝试让用户直接针对输出提问，再由系统呈现与该问题有关的运行时因果链。

研究问题 研究问题一（RQ1）：用户是否认为基于“为什么”与“为什么没有”提问的 Whyline 有用？

研究问题二（RQ2）：Whyline 能否缩短调试时间并帮助程序员完成更多任务？

研究方法 作者在 Alice 编程环境中实现 Whyline，并对既有无 Whyline 研究和新增有 Whyline 研究中的相同场景进行比较。无工具组 4 人、有工具组 5 人，在 90 分钟内完成 6 项 Pac-Man 编程任务并出声思考。

主要发现 在 6 个可比调试场景中，Whyline 把调试时间平均缩短 7.8 倍；有工具组平均完成 3.20 项任务，无工具组为 2.25 项，提升约 40%。19/24 次提问中工具被认为有用，“为什么没有”问题又明显多于“为什么”问题。

贡献与意义 Whyline 把可见程序输出转成问题菜单，并用时间游标、因果箭头和可逆历史把现象、运行事件与代码连接起来，改变了从代码位置开始的调试顺序。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：07

名称 Finding Causes of Program Output with the Java Whyline

作者 Andrew J. Ko; Brad A. Myers 译者

出处 【美国】Proceedings of the 27th ACM Conference on Human Factors in Computing Systems, 2009: 1569-1578 页

文献阅读心得（每篇不少于 150 字）：

研究背景 大型 Java 程序中，断点调试要求开发者先选对代码位置，而用户通常更容易识别界面中出现或缺失的输出。Java Whyline 把“从结果反查原因”的思路扩展到真实开源项目。

研究问题 研究问题一（RQ1）：Java Whyline 是否比断点调试器更快、更成功地帮助开发者定位真实缺陷？

研究问题二（RQ2）：两种工具如何改变开发者浏览文件、使用问题和接近关键函数的路径？

研究方法 研究采用组间实验，两组各 10 名软件工程硕士生，分别使用 Whyline 或具有相同外观的断点调试器，诊断 ArgoUML 的两个真实缺陷。每项任务限时 30 分钟，并记录成功率、时间、文件访问和导航距离。

主要发现 任务 1 中 Whyline 组 10/10 成功，对照组 3/10，且前者约快 2 倍；任务 2 中分别为 4/10 和 0/10。总体上 Whyline 用户约有 3 倍成功率，并更快接近关键代码；但工具更善于找到故障方法，不一定能解释方法为何错误。

贡献与意义 研究把输出选择、执行录像、依赖可视化、书签与问题导航集成为可实证比较的调试界面，并使用真实大型项目而非人工小程序。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：08

名称 How Programmers Debug, Revisited: An Information Foraging Theory Perspective

作者 Joseph Lawrance; Christopher Bogart; Margaret Burnett; Rachel Bellamy; Kyle Rector; Scott D. Fleming 译者

出处 【美国】IEEE Transactions on Software Engineering, 2013, 39(2): 197-215 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试大型代码库时，程序员不断判断下一条线索是否值得追踪。仅用“提出并验证假设”描述这一过程，很难解释他们在文件、方法和调用关系之间的实际导航，也难为工具设计提供直接建议。

研究问题 研究问题一（RQ1）：专业程序员调试时的语言和导航是否主要体现信息气味追踪，而非显式假设？

研究问题二（RQ2）：信息气味与代码拓扑模型能否预测程序员下一步会访问哪里？

研究方法 作者提出基于信息觅食理论的调试模型，并观察 10 名专业程序员调试真实开源程序，分析出声思考与导航日志；随后构建可执行模型，与不考虑信息气味的替代模型比较预测能力。

主要发现 参与者的言语更多涉及寻找和比较线索，而不是陈述完整假设。错误信息、标识符、文件名和调用关系形成信息气味；纳入气味的模型更准确地预测导航，代码拓扑则决定线索间移动成本。

贡献与意义 论文以信息气味（scent）和代码拓扑（topology）两个可观测、可计算的构念解释调试导航，减少对难以直接测量的心理假设的依赖，并把理论转成可执行预测模型。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：09

名称 Mental Procedures in Real-Life Tasks: A Case Study of Electronic Trouble Shooting

作者 Jens Rasmussen; A. Jensen 译者

出处 【英国】Ergonomics, 1974, 17(3): 293-307 页

文献阅读心得（每篇不少于 150 字）：

研究背景 熟练维修人员面对电子设备故障时，不会穷举所有测量，而是在系统、功能单元和部件组成的层级结构中搜索。真实工作还要求兼顾诊断时间、认知负荷和测量成本。

研究问题 研究问题一（RQ1）：熟练电子维修人员如何把设备表示为层级系统并组织故障搜索？

研究问题二（RQ2）：不同搜索例程在观察数量、知识深度和心理加工复杂度上有什么差异？

研究方法 研究在维修人员的正常工作环境中收集口述协议，对真实电子排故过程进行细致编码，分析症状判断、层级切换、测量选择及反复出现的搜索例程。

主要发现 排故可分解为多种循环搜索例程，分别依赖不同深度的内部机理知识。维修人员偏好快速连续的好/坏判断，即使某些观察在信息论上冗余；若目标是减少时间和心理负担，这种做法仍然合理。

贡献与意义 论文较早把真实排故中的认知过程描述为层级系统上的搜索，并揭示“信息最省”与“时间、心智成本最优”并不相同。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：10

名称 Making Sense of Sensemaking 1: Alternative Perspectives

作者 Gary Klein; Brian Moon; Robert R. Hoffman 译者

出处 【美国】IEEE Intelligent Systems, 2006, 21(4): 70-73 页

文献阅读心得（每篇不少于 150 字）：

研究背景 意义建构（sensemaking）在心理学、组织研究、自然情境决策和人机交互（human-computer interaction, HCI）中有多种含义。复杂任务中的理解不是对信息做一次摘要，而是持续寻找联系、解释意外并为下一步行动建立可用认识。

研究问题 研究问题一（RQ1）：不同研究传统如何界定意义建构，它与理解、情境意识等概念有何关系？

研究问题二（RQ2）：哪种定义能解释人在不确定环境中为了预测和行动而持续组织联系？

研究方法 本文是概念综述与立场论文。作者比较多个学科的定义和代表性案例，讨论意义建构的触发条件、目的与分析尺度，并为后续宏认知模型建立概念边界。

主要发现 作者强调意义建构是一种有动机、持续的努力：理解人、地点和事件之间的联系，以预测其发展并有效行动。它常由惊讶、冲突或知识缺口触发，结果是可行动的解释，而非静态信息表征。

贡献与意义 论文跨学科澄清了意义建构的不同视角，把“联系—预测—行动”作为共同核心，为研究真实复杂工作中的理解活动提供了统一入口。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：11

名称 Making Sense of Sensemaking 2: A Macrocognitive Model

作者 Gary Klein; Brian Moon; Robert R. Hoffman 译者

出处 【美国】IEEE Intelligent Systems, 2006, 21(5): 88-92 页

文献阅读心得（每篇不少于 150 字）：

研究背景 人在不确定任务中会借助已有框架筛选数据，数据又可能迫使框架改变。把意义建构理解成固定步骤，会忽略专家在解释、注意分配、反馈和行动之间的往返。

研究问题 研究问题一（RQ1）：数据与解释框架如何相互依存并推动意义建构？

研究问题二（RQ2）：人何时会扩展、质疑、比较、保留或重构当前框架？

研究方法 作者基于自然情境决策研究提出数据—框架（Data/Frame）宏认知模型，并用因果推理、假设承诺、反馈学习、专家差异和确认偏差等经验研究检验模型含义。

主要发现 框架管理注意并定义、连接和过滤数据；异常与不一致可触发质疑或重构。过早形成框架并非必然有害，适度承诺能产生可检验预期；过程反馈通常比只告知成败的结果反馈更有助于改进。

贡献与意义 论文以双向的数据—框架循环替代线性“先收集数据、后得出解释”，并提出连接、扩展、质疑、比较、重构和寻找框架等可观察活动。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：12

名称 A Data-Frame Theory of Sensemaking

作者 Gary Klein; Jennifer K. Phillips; Erica L. Rall; Deborah A. Peluso 译者

出处 【美国】Expertise Out of Context, Mahwah, NJ: Lawrence Erlbaum Associates, 2007: 113-155 页

文献阅读心得（每篇不少于 150 字）：

研究背景 传统微观认知研究多解释人如何理解单词、句子等简单刺激，难以覆盖真实工作中对模糊、矛盾、动态信息形成可行动理解的过程。数据—框架理论（Data-Frame theory）由此转向自然情境中的宏观意义建构。

研究问题 研究问题一（RQ1）：数据如何被框架选择和解释，框架又如何由少量关键锚点被识别或建构？

研究问题二（RQ2）：当数据与框架不一致时，意义建构有哪些可能形式与转换路径？

研究方法 本章综合自然情境决策、专业知识和问题解决研究，借助真实事件材料提出 9 条理论命题，并系统描述连接数据与框架、扩展、质疑、保留、比较、重构和寻找新框架等活动。

主要发现 数据不是完全中性的输入，而是在框架中被辨认和赋义；框架也由关键线索推断。意义建构通常在二者达到足够一致时停止，并服务于“接下来做什么”的功能性理解。专家与新手主要差在可调用框架的丰富度。

贡献与意义 理论同时说明数据驱动和框架驱动过程，明确列出多种非线性活动及其触发关系，可用于分析解释如何被建立、维护和推翻。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：13

名称 AgentGUI: An Interface for Observing and Steering Long-Running AI Agents

作者 Xuan Zhao; Jiwoong Sohn; Qinyue Zheng; Michael Moor 译者

出处 【瑞士】arXiv preprint arXiv:2607.26300, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 长时运行的智能体可同时执行多个任务，但人类监督仍依赖终端输出和分散日志。用户需要在不中断整体工作的情况下掌握轨迹、产物、成本和偏移，并在必要时实施人工或自动纠偏。

研究问题 研究问题一（RQ1）：统一的轨迹可视化能否提高用户理解长时智能体执行的速度和准确性？

研究问题二（RQ2）：人工与自动的管理者式引导（steering）能否在任务漂移时改善最终完成率？

研究方法 系统整合多会话轨迹、工作区快照、人工输入和自动管理模块。8 名硕博参与者在被试内、顺序平衡实验中比较 AgentGUI 与 Hermes 仪表盘（Hermes Dashboard）；另对多种模型各运行 50 次以测试自动纠偏。

主要发现 参与者用 AgentGUI 回答轨迹问题平均快 38%（90 秒对 145 秒， $p = .023$ ），准确率为 93% 对 80%（ $p = .031$ ），且没有速度—准确率交换；自动管理模块在部分模型上将完成率提高最多 34 个百分点。

贡献与意义 论文把并行会话、轨迹理解、工作区产物、人工指令和自动审计放进同一桌面式界面，并用客观理解速度、准确率和 workload 评价可观测性。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：14

名称 An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks

作者 Amy J. Ko; Brad A. Myers; Michael J. Coblenz; Htet Htet Aung 译者

出处 【美国】IEEE Transactions on Software Engineering, 2006, 32(12): 971-987, DOI: 10.1109/TSE.2006.116 页

文献阅读心得（每篇不少于 150 字）：

研究背景 软件维护不仅需要读懂局部代码，还要在多文件、多工具和多任务之间寻找、关联并暂存相关信息。现代集成开发环境（integrated development environment, IDE）的导航机制可能既提供线索，也制造误导与重复切换。

研究问题 研究问题一（RQ1）：开发者在维护任务中如何搜索、关联和收集相关代码与运行信息？

研究问题二（RQ2）：集成开发环境的线索、依赖导航和交互状态如何支持或妨碍这些活动？

研究方法 研究从 31 名 Java 开发者中选取经验最高的 10 名，要求其在 70 分钟内完成同一 503 行 Java Paint 程序的 5 个维护任务，并接受定时中断。作者逐帧分析屏幕录像、语音和交互日志。

主要发现 开发者平均 35% 的有效时间用于源文件内外的导航，许多跳转是重复且本可避免的；失败搜索常由误导性信息气味开始。记忆不可靠、纸笔缓慢、集成开发环境的交互状态易被后续任务覆盖，均难稳定保存任务上下文。

贡献与意义 论文提出“搜索—关联—收集”的程序理解模型，将信息觅食与集成开发环境的具体交互结合，并以真实多任务和中断条件揭示导航成本及外部记忆需求。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：15

名称 Debugging: An Analysis of Bug-Location Strategies

作者 Irvin R. Katz; John R. Anderson 译者

出处 【美国】Human-Computer Interaction, 1987-1988, 3(4): 351-399, DOI: 10.1207/S15327051HCI0304_2 页

文献阅读心得（每篇不少于 150 字）：

研究背景 具备语言知识的程序员往往能在看到错误行后修正它，真正困难的是从程序行为中定位哪一行有错。调试策略可能因代码归属、可用线索和搜索顺序而改变。

研究问题 研究问题一（RQ1）：熟悉表处理语言（LISP）的学生在定位程序错误时采用哪些策略，错误更接近知识缺陷还是偶发失误？

研究问题二（RQ2）：调试自己的程序与他人程序是否改变定位策略，策略又如何影响检查与修复？

研究方法 论文报告 4 项表处理语言调试实验，比较错误重复性、自然使用的定位策略、自编与他编程序情境，以及不同策略对被检查代码行和后续判断修复的影响。

主要发现 参与者的错误多为不易重复的 slip，通常一旦定位即可修复；他们会使用多种错误定位策略，且自编与他编程序的策略选择不同。策略主要改变搜索哪些代码行，对选中一行后的正确性判断和修复能力影响较小。

贡献与意义 研究把“找错位置”与“判断并修复错误”分解为不同阶段，并通过连续实验说明调试瓶颈并非总是缺少编程知识，而是搜索与定位。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：16

名称 Theory of Troubleshooting: The Developer's Cognitive Experience of Overcoming Confusion

作者 Arty Starr; Margaret-Anne Storey 译者

出处 【美国】ACM Transactions on Software Engineering and Methodology, 2026, DOI: 10.1145/3800945; arXiv:2602.10540 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试研究常聚焦工具和步骤，却较少解释开发者面对意外行为时的内部体验。长时间困惑会持续占用注意、工作记忆和心理模型构建资源，并可能转化为疲劳与项目可持续性风险。

研究问题 研究问题一（RQ1）：开发者在排查意外系统行为时，思考、感受和努力随时间如何变化？

研究问题二（RQ2）：困惑、经验直觉、试探性实验、进展与认知疲劳之间存在怎样的关系？

研究方法 作者访谈 27 名职业开发者，合计 570 年经验，并开展 7 次回访；采用建构主义扎根理论，从 681 条扎根编码形成 8 个核心理论类别，在前 12 份访谈后达到理论饱和，并以参与者反馈检验共鸣。

主要发现 排故从困惑体验启动，经验直觉引导策略，开发者通过“试一试、看结果”缩小解释空间；获得正或负知识都可被体验为进展。持续无进展会耗竭注意并引发补偿努力、盲视和疲劳，最终在“想通了”时解除。

贡献与意义 论文把排故定义为识别、理解并建立意外行为原因心理模型的认知过程，用连贯理论连接情绪、注意、实验、直觉和疲劳，而不只描述技术操作。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：17

名称 The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis

作者 Peter Pirolli; Stuart K. Card 译者

出处 【美国】 Proceedings of the 2005 International Conference on Intelligence Analysis, 2005 页

文献阅读心得（每篇不少于 150 字）：

研究背景 情报分析需要从大量异质材料形成结构、假设和可交付结论，但规范性流程很少说明专家真实工作中如何往返搜索、筛选、重组和验证证据。

研究问题 研究问题一（RQ1）：专家分析者如何在信息觅食与意义建构之间循环，将信息转化为模式、洞见和产品？

研究问题二（RQ2）：认知任务分析揭示了哪些可由交互技术降低成本的关键杠杆点？

研究方法 作者采用认知任务分析、分析者访谈和出声思考协议，结合专家图式与信息觅食理论，构建从外部数据源到证据文件、图式、假设和最终报告的双环模型。

主要发现 分析并非线性阅读，而是在探索、富集、利用与表示重构之间机会主义往返；常见外部结构包括资料箱（shoebox）、证据文件（evidence file）和图式（schema）。高成本环节集中在搜索、筛选、重新表示、比较假设和组装证据。

贡献与意义 论文将信息觅食循环（foraging loop）与意义建构循环（sensemaking loop）连接起来，并从每个转换的时间和注意成本推导工具设计杠杆，为总览—筛选—证据—假设界面提供理论基础。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：18

名称 AgentStepper: Interactive Debugging of Software Development Agents

作者 Robert Hutter; Michael Pradel 译者

出处 【德国】arXiv preprint arXiv:2602.06593, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 软件开发智能体会在模型请求、工具调用和代码修改之间形成长轨迹。原始日志虽然保留了大量信息，却缺少适合开发者理解高层行为的结构，也不能像传统调试器那样暂停、单步和检查中间状态。

研究问题 研究问题一（RQ1）：将传统调试中的断点、单步执行和状态检查提升到智能体动作层，能否帮助开发者理解智能体轨迹？

研究问题二（RQ2）：AgentStepper 接入现有软件开发智能体需要多少改动，它是否改善轨迹理解、智能体程序缺陷识别和主观负荷？

研究方法 作者把 AgentStepper 接入 ExecutionAgent、SWE-Agent 和 RepairAgent，统计集成改动量；随后邀请 12 名参与者完成一项轨迹理解任务和两项智能体调试任务，比较 AgentStepper 与自行选择的常规日志工具。

主要发现 三种智能体各需插入 5—7 个应用程序编程接口（application programming interface, API）调用并修改 39—42 行代码。AgentStepper 条件下轨迹理解平均表现由 64% 升至 67%，智能体程序缺陷识别成功率由 17% 升至 60%，挫败感由 5.4/7 降至 2.4/7。

贡献与意义 论文把智能体轨迹表示为“模型—智能体程序”和“智能体程序—工具”两条交织会话，并整合总览、细节、代码变更和断点控制。它是本研究原型最直接的人侧调试先例，也说明“理解轨迹”和“定位运行框架实现缺陷”应分开测量。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：19

名称 AgentSight: System-Level Observability for AI Agents Using eBPF

作者 Yusheng Zheng; Yanpeng Hu; Tong Yu; Andi Quinn 译者

出处 【美国】arXiv preprint arXiv:2508.02736, 2025 页

文献阅读心得（每篇不少于 150 字）：

研究背景 智能体的高层意图存在于模型提示和响应中，实际效果却表现为进程、文件和网络事件。现有工具通常只能看到其中一侧，导致开发者无法判断某个系统行为是否由智能体意图、工具实现或外部环境造成。

研究问题 研究问题一（RQ1）：能否在不修改智能体框架代码的情况下关联模型层语义意图与系统层实际行为？

研究问题二（RQ2）：这种跨层观测能否以可接受的开销识别安全攻击、低效循环和多智能体协调问题？

研究方法 作者提出边界追踪（boundary tracing），利用扩展伯克利包过滤器（extended Berkeley Packet Filter, eBPF）监视内核事件，同时截获传输层安全协议（Transport Layer Security, TLS）加密的模型流量以提取语义意图，再通过实时关联引擎和辅助大语言模型跨进程连接两类事件，并在多种攻击和故障场景中评估。

主要发现 AgentSight 能够检测提示注入、识别浪费资源的推理循环并发现隐藏的多智能体协调瓶颈，运行开销低于 3%。研究也表明，系统级事实和语义解释来自不同观测通道。

贡献与意义 论文具体展示了跨层证据如何被关联，并揭示“直接记录的系统事件”和“经过模型分析得到的语义关系”不能混为同一种证据。它为本研究原型的记录、推导和未知三类证据状态提供了技术边界。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：20

名称 From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws

作者 Mengzhuo Chen; Junjie Wang; Zhe Liu; Yawen Wang; Haiming Zheng; Qing Wang 译者

出处 【中国】arXiv preprint arXiv:2606.06324v2, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 智能体失败往往由执行环境、工具、上下文、生命周期、可观测、验证或治理机制共同塑造。仅依据最终成败调整提示词，难以确定失败证据位于哪一步，也难以把运行现象映射到具体运行框架工件。

研究问题 研究问题一（RQ1）：如何把碎片化运行轨迹编译成能够表达步骤级数据流、控制流和运行框架实现锚点的统一表示？

研究问题二（RQ2）：基于该表示形成的局部诊断和范围化修复，能否在不引入不可接受回归的情况下改善智能体表现？

研究方法 HarnessFix 将原始轨迹与运行框架工件编译为面向运行框架的轨迹中间表示（Harness-aware Trace Intermediate Representation, HTIR），定位责任步骤和实现工件，把重复诊断合并为缺陷记录，再按缺陷类型生成补丁并做回归约束验证；实验覆盖四个公开基准。

主要发现 相较初始运行框架，HarnessFix 在四个基准上的性能提高 6.3 至 18.4 个百分点，并优于人工设计和自演化基线。结果显示，失败轨迹可被组织成指向具体运行框架机制的结构化证据。

贡献与意义 论文证明数据流、控制流和实现锚点可以同时进入面向运行框架的轨迹表示，是与本研究原型最接近的技术工作。其自动修复不是本研究的目标，但 HTIR 所物化的关联类型构成了人侧理解界面的关键比较对象。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：21

名称 Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses

作者 Jiahang Lin; Shichun Liu; Chengjun Pan; Lizhi Lin; Shihan Dou; Zhiheng Xi; Xuanjing Huang; Hang Yan; Zhenhua Han; Tao Gui; Yu-Gang Jiang 译者

出处 【中国】arXiv preprint arXiv:2604.25850v4, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 智能体运行框架包含工具、中间件、长期记忆、系统提示和运行规则，修改空间异构，轨迹规模巨大，而且一次修改与下一轮结果之间的关系难以归因。自动演化若缺少可观测证据，容易退化为试错。

研究问题 研究问题一（RQ1）：组件、运行经验和修改决策需要怎样的可观测表示，才能形成可验证的运行框架演化闭环？

研究问题二（RQ2）：可观测性驱动自动演化能否提高编码智能体性能，并跨模型迁移而非只适配单一基准？

研究方法 作者提出组件可观测、经验可观测和决策可观测三支柱，在 Terminal-Bench 2 上执行 10 轮运行框架演化，并通过消融和跨模型、跨基准迁移检验各类组件修改的作用。

主要发现 Terminal-Bench 2 的单次通过率（pass@1）由 69.7% 升至 77.0%；冻结后的运行框架在其他模型家族上带来 5.1—10.1 个百分点提升，并以比种子运行框架少 12% 的词元（token）取得 SWE-bench Verified 上的领先结果。增益主要来自工具、中间件和长期记忆，而非系统提示。

贡献与意义 论文把可观测性从日志采集提升为组件、经验和决策三个层次，并要求每次修改都有可证伪预测。它说明本研究不能只研究提示词，而应将上下文和工具等机制放入统一运行框架结构。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：22

名称 AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems

作者 Jiaying Lu; Bo Pan; Jieyi Chen; Yingchaojie Feng; Jingyuan Hu; Yuchen Peng; Wei Chen 译者

出处【美国】IEEE Transactions on Visualization and Computer Graphics, 2025, 31(8): 4182-4197, DOI: 10.1109/TVCG.2024.3394053 页

文献阅读心得（每篇不少于 150 字）：

研究背景 多智能体自治系统产生大量随时间演化的事件、记忆和交互。线性日志难以呈现跨时间行为模式，也难以让用户从总体演化下钻到单个智能体行为及其可能原因。

研究问题 研究问题一（RQ1）：如何把原始执行事件组织成支持多粒度查看的智能体行为模型，并追踪行为之间的潜在因果关系？

研究问题二（RQ2）：层级时间视图、单智能体细节视图和环境监视视图能否支持用户发现并解释复杂行为？

研究方法 作者建立事件处理、行为摘要和原因追踪流程，开发包含概览（Outline）、智能体（Agent）和监视（Monitor）三类联动视图的 AgentLens，并通过两个使用场景和用户研究评价系统。

主要发现 参与者能够沿层级时间线发现行为模式、下钻检查事件，并回看可能影响当前行为的历史操作。系统有效性和可用性得到验证，但部分因果关系依据文本相似度推断，并非直接观测事实。

贡献与意义 论文提供总览—细节联动和原因追踪的成熟视觉设计，也暴露了本研究原型要解决的风险：界面若不说明连线来自日志、算法推断还是缺失信号，使用者可能把潜在关系理解为实际因果。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：23

名称 AgentGraph: Trace-to-Graph Platform for Interactive Analysis and Robustness Testing in Agentic AI Systems

作者 Zekun Wu; Seonglae Cho; Cristian Enrique Munoz Villalobos; Theo King; Umar Mohammed; Emre Kazim; Maria Perez-Ortiz; Sahan Bulathwela; Adriano Koshiyama
译者

出处 【美国】Proceedings of the AAAI Conference on Artificial Intelligence, 2026, 40(48): 41721-41723, DOI: 10.1609/aaai.v40i48.42393 页

文献阅读心得（每篇不少于 150 字）：

研究背景 现有智能体可观测平台多记录提示输入输出和运行指标，却仍要求用户手工从线性轨迹恢复任务、工具、数据和人员之间的结构关系。

研究问题 研究问题一（RQ1）：如何把智能体执行日志转换为带类型节点和边的交互知识图，并保持每个图元素可回到原始轨迹？

研究问题二（RQ2）：统一图表示能否同时支持故障分析、优化建议和扰动条件下的稳健性测试？

研究方法 AgentGraph 把智能体、任务、工具、数据输入输出和人建模为节点，用消费、委托、顺序、工具使用、输出交付和人工干预等类型边连接，并让每个元素指向精确轨迹片段（trace span）；系统还加入扰动和因果归因分析。

主要发现 原型能够在同一图中开展基于轨迹的定性故障分析和定量稳健性测试。论文属于演示轨道，主要证明系统能力，尚未提供充分的人类理解效果证据。

贡献与意义 论文将关系图与精确原始证据链接，是本研究原型“关联必须可核验”的直接先例；同时也说明系统功能展示不能替代对理解正确率、误判率和无证据结论的受控评估。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：24

名称 Graph of Trace: Visualizing Execution Traces of Scientific Agents

作者 Tianci Gao; Haoxuan Li; Jian He Li; Tianxiang Zhao; Shi Runze; Weiran Wang; Zezhao Wu; Lu Mi 译者

出处 【美国】Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, 2026:297-306, DOI: 10.18653/v1/2026.acl-demo.29 页

文献阅读心得（每篇不少于 150 字）：

研究背景 科学智能体可以自动执行环境配置、代码运行、数据分析和结论生成，但展开后的工作流难以供领域专家检查，限制了结果复核和后续人机协作。

研究问题 研究问题一（RQ1）：如何实时记录细粒度执行事件并把科学智能体工作流组织成可检查的有向图？

研究问题二（RQ2）：结构化轨迹图是否改善领域专家对工作流、失败位置和结果生成过程的理解？

研究方法 系统记录工具调用、代码执行和中间产物，由监视模块构建实时更新的有向图，并提供对话、图和节点细节检查器；作者在复杂科研任务上邀请人工智能、神经科学和生物学专家评价。

主要发现 专家认为结构化轨迹提高了工作流理解、感知可解释性和进一步分析交互的可用性。评价以专家反馈为主，尚未通过同证据基线比较理解准确率和无依据判断。

贡献与意义 论文提供与本研究原型相近的实时轨迹图和总览一节点细节结构，同时构成重要方法学对照：本研究需要把主观可用性扩展为具有客观真值（ground truth）的理解和证据判断实验。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：25

名称 AUTOGEN STUDIO: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems

作者 Victor Dibia; Jingya Chen; Gagan Bansal; Suff Syed; Adam Fourney; Erkang Zhu; Chi Wang; Saleema Amershi 译者

出处 【美国】Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2024: 72 - 79, DOI: 10.18653/v1/2024.emnlp-demo.8 页

文献阅读心得（每篇不少于 150 字）：

研究背景 多智能体 workflow 涉及模型、工具、角色和编排参数，直接通过代码配置和日志排查门槛较高。开发者需要同时查看系统声明结构和具体运行结果。

研究问题 研究问题一（RQ1）：声明式规范和无代码界面如何降低多智能体 workflow 构建、调试和评价的复杂度？

研究问题二（RQ2）：哪些界面原则能够让模型、工具、智能体和编排关系被复用并在运行中检查？

研究方法 作者基于 AutoGen 开发网页（Web）界面和 Python 应用程序编程接口，以 JavaScript 对象表示法（JavaScript Object Notation, JSON）声明智能体及 workflow，提供拖放式结构编辑、交互运行与调试、结果评估和可复用组件库，并通过系统案例说明设计。

主要发现 AutoGen Studio 把 workflow 结构、组件配置和运行查看统一到一个开发环境中，支持快速原型和非代码式调试。论文主要是系统演示，没有隔离不同呈现方式对理解正确率的影响。

贡献与意义 论文证明“声明结构”和“运行轨迹”可以在同一开发工具中组织，并提供可复用的智能体系统表达方式；本研究进一步关心静态声明与本次运行事实之间的证据差异。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：26

名称 AgentTrace: A Structured Logging Framework for Agent System Observability

作者 Adam AlSayyad; Kelvin Yuxiang Huang; Richik Pal 译者

出处 【美国】arXiv preprint arXiv:2602.10133, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 传统日志偏向操作事件，难以同时表达智能体的模型交互、环境动作和上下文状态。静态审计也难以应对由模型推断产生的非确定行为。

研究问题 研究问题一（RQ1）：智能体运行时日志应覆盖哪些观测面，才能支持调试、安全审计和责任追踪？

研究问题二（RQ2）：结构化日志能否在较少侵入智能体代码的情况下接入现有遥测基础设施？

研究方法 作者提出操作（operational）、认知（cognitive）和上下文（contextual）三类日志面，设计图式化（schema-based）运行时记录机制，并展示其与开放遥测框架（OpenTelemetry）等基础设施的集成方式。

主要发现 框架能够持续记录工具和环境动作、模型相关状态以及上下文信息，为实时监控和事后检查提供共同结构。论文以框架设计为主，尚未验证开发者如何理解这些日志。

贡献与意义 论文把上下文状态明确列为与操作和认知并列的观测面，直接支持本研究聚焦上下文构建与变换；同时提醒研究区分“记录结构完整”与“人能正确理解”两类贡献。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：27

名称 From Agent Traces to Trust: Evidence Tracing and Execution Provenance in LLM Agents

作者 Yiqi Wang; Jiaqi Zhang; Taotao Cai; Zirui Liu; Qingqiang Sun; Zequn Sun; Zhangkai Wu; Mingkai Zhang; Yanming Zhu 译者

出处 【中国】arXiv preprint arXiv:2606.04990, 2026 页

文献阅读心得（每篇不少于 150 字）：

研究背景 最终答案正确与否不能说明证据如何进入结论、工具调用是否合理、记忆怎样影响后续决策，也不能定位执行失败。相关工作分散在检索归因、工具安全、记忆、可观测和调试领域。

研究问题 研究问题一（RQ1）：大语言模型智能体中的证据单元、执行单元和溯源关系可以怎样统一分类？

研究问题二（RQ2）：现有方法在表示、粒度、时间、评价和信任功能上有哪些覆盖与缺口？

研究方法 作者开展系统性综述，从轨迹（trace）来源、证据与执行单元、溯源（provenance）关系、追踪粒度与时机、表示形式和信任功能建立统一分类，并映射数据集、基准和评价指标。

主要发现 现有研究能够追踪检索、工具、记忆和动作的部分来源，但缺少统一的轨迹图式（trace schema）、语义级溯源、真实执行基准和面向恢复的过程评价，隐私与审计基础设施也不成熟。

贡献与意义 论文直接把证据追踪与执行溯源连接起来，为本研究定义“关联证据”提供最新综述框架，也支持把评价从最终答案扩展到过程可核验性和证据边界。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：28

名称 LADYBUG: an LLM Agent DeBUGger for Data-Driven Applications

作者 Joel Rorseth; Parke Godfrey; Lukasz Golab; Divesh Srivastava; Jarek Szlichta
译者

出处 【荷兰】Proceedings of the 28th International Conference on Extending Database Technology, 2025: 1082-1085, DOI: 10.48786/EDBT.2025.94 页

文献阅读心得（每篇不少于 150 字）：

研究背景 数据驱动智能体常以多步工作流生成代码、转换数据并汇总结果。某一步的隐蔽错误可能不触发异常，却污染后续结果，用户需要检查中间输入输出并限定重新执行范围。

研究问题 研究问题一（RQ1）：交互式时间线能否支持用户检查智能体每一步的输入、输出和代码，并在任意步骤介入？

研究问题二（RQ2）：大语言模型辅助调试器如何在用户难以发现隐蔽错误时提出候选错误步骤和干预？

研究方法 作者实现 LADYBUG 演示系统，把智能体执行表示为按时间排列的步骤，允许用户查看中间状态、修改任意步骤并只重算受影响部分，同时加入基于自我反思的大语言模型辅助调试器。

主要发现 系统能在示例数据应用中定位并修正未直接报错的中间错误，并通过依赖关系减少重算范围。论文是演示短文，没有人类对照实验，也不能把大语言模型提出的原因视为已证实事实。

贡献与意义 论文展示了步骤、依赖和局部重算如何共同支持智能体调试。对本研究而言，其更重要的意义是区分可直接检查的步骤事实、系统计算的依赖关系和大语言模型提出的候选解释。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：29

名称 Open Agent Specification: Enabling Cross-Framework Comparison of AI Agents

作者 Soufiane Amini; Yassine Benajiba; Cesare Bernardis; Paul Cayet; et al. 译者

出处 【美国】Proceedings of the ACM Conference on AI and Agentic Systems, 2026: 404-418, DOI: 10.1145/3786335.3813130 页

文献阅读心得（每篇不少于 150 字）：

研究背景 不同智能体框架具有不同的提示处理、工具适配、编排和执行语义。即使智能体设计看似相同，运行时也可能产生不同路径和结果，导致跨框架比较缺少共同参照。

研究问题 研究问题一（RQ1）：如何用统一声明表示固定的智能体结构、数据流和控制流，并在不同运行时中生成可比执行？

研究问题二（RQ2）：相同智能体规范在不同框架中是否仍会出现准确率、时延和执行行为差异？

研究方法 作者提出框架无关的智能体规范（Agent Specification）及标准化评估运行框架，在 LangGraph、CrewAI、AutoGen 和 WayFlow 四种运行时上执行 SimpleQA Verified、 τ^2 -Bench 和 BIRD-SQL 三个基准，并记录统一结构的运行轨迹（trace）。

主要发现 在共享规范下，不同运行时仍表现出显著的准确率、时延和执行差异，说明框架适配器、提示和编排实现会塑造运行行为，不能由静态规范直接推断。

贡献与意义 论文为结构与运行的对照提供了统一表示和跨框架证据，直接支撑本研究区分“预设结构”与“本次实际执行”；它也为构造相同任务、不同运行框架机制的实验材料提供方法。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：30

名称 X-Trace: A Pervasive Network Tracing Framework

作者 Rodrigo Fonseca; George Porter; Randy H. Katz; Scott Shenker; Ion Stoica 译者

出处 【美国】Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation, 2007: 271-284 页

文献阅读心得（每篇不少于 150 字）：

研究背景 复杂网络服务跨越应用、协议和管理域，单层日志不能重建一次用户任务经过的完整路径。即使各组件都产生日志，也缺少将分散事件关联到同一任务的机制。

研究问题 研究问题一（RQ1）：如何在跨层、跨应用执行中传播统一任务标识并重建因果任务树？

研究问题二（RQ2）：该机制在 DNS、Web 和覆盖网络等异构系统中能否提供可用于诊断的端到端视图？

研究方法 X-Trace 在初始任务中注入元数据，并沿协议调用、转发和递归请求传播；各组件记录带共同任务标识的事件，系统据此重建任务树，并在 DNS、三层网站和覆盖网络场景中部署。

主要发现 跨组件任务树能把原本分散的日志连接起来并定位错误数据或异常路径；即使只在部分层部署，仍可提供局部诊断价值，但未被记录的分支不能被当作未发生。

贡献与意义 论文奠定了端到端执行溯源的任务树思想，说明可靠关联依赖显式传播的记录机制。它为本研究界定“直接记录”提供技术基础，也说明信号缺失应标为未知。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：31

名称 Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems

作者 Jonathan Mace; Ryan Roelke; Rodrigo Fonseca 译者

出处 【美国】Proceedings of the 25th ACM Symposium on Operating Systems Principles, 2015: 378-393, DOI: 10.1145/2815400.2815415 页

文献阅读心得（每篇不少于 150 字）：

研究背景 分布式系统中的日志和指标通常预先定义且以单组件为中心，发生未知问题时，开发者难以在运行中提出新的跨组件问题，也难以关联不同位置的事件。

研究问题 研究问题一（RQ1）：能否动态定义监测查询，并在跨机器和跨组件事件之间保留因果关系？

研究问题二（RQ2）：动态因果监测能否以低开销定位软件缺陷、错误配置和性能退化等不同根因？

研究方法 作者结合动态插桩与先发生关系连接（happened-before join），使用户可在系统某处定义指标，却按其他组件中的因果先行事件选择、过滤和分组；原型在包含 HDFS、HBase、MapReduce 和 YARN 的 Hadoop 集群中评估。

主要发现 Pivot Tracing 能够跨层查询并定位软件错误、配置问题和慢硬件等多类根因，同时保持较低运行开销。它表明关系可以由明确的因果传播规则计算，而不一定逐条直接记录。

贡献与意义 论文为本研究的“按规则推导”提供清晰先例：关系有可检查规则和输入证据，但仍不同于原生日志事实。界面应把推导方法、覆盖范围和无法关联的事件一起呈现。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：32

名称 Dapper, a Large-Scale Distributed Systems Tracing Infrastructure

作者 Benjamin H. Sigelman; Luiz André Barroso; Mike Burrows; Pat Stephenson; Manoj Plakal; Donald Beaver; Saul Jaspan; Chandan Shanbhag 译者

出处 【美国】Google Technical Report dapper-2010-1, 2010 页

文献阅读心得（每篇不少于 150 字）：

研究背景 大规模互联网服务由不同团队、语言和机器上的组件组成。开发者需要理解一次请求跨组件的路径和耗时，但全面插桩会带来开发负担、运行开销与海量数据。

研究问题 研究问题一（RQ1）：如何在低开销、应用透明和大规模持续部署之间取得平衡，记录端到端请求轨迹？

研究问题二（RQ2）：采样式分布追踪是否能支持开发和运维团队理解系统行为并构建多种诊断工具？

研究方法 Dapper 在少量公共库中插桩，以追踪标识（trace）和跨度标识（span）记录请求层级及时间信息，通过采样控制开销；论文总结系统在 Google 内部两年多的构建、部署、使用和衍生工具经验。

主要发现 公共库插桩和采样使追踪能够在超大规模生产环境持续运行，并支持性能分析、依赖发现和异常诊断。但采样和未插桩区域意味着轨迹天然不完整。

贡献与意义 论文建立了现代分布式追踪的追踪/跨度标识基础，并揭示观测证据的覆盖边界。对本研究而言，界面必须避免把被采样或未插桩造成的空白解释成机制没有运行。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：33

名称 A Controlled Experiment for Program Comprehension through Trace Visualization

作者 Bas Cornelissen; Andy Zaidman; Arie van Deursen 译者

出处 【美国】IEEE Transactions on Software Engineering, 2011, 37(3): 341-355,
DOI: 10.1109/TSE.2010.47 页

文献阅读心得（每篇不少于 150 字）：

研究背景 执行轨迹可视化常被用于程序理解，但早期工具多依赖案例演示，缺少把可视化与常规开发环境直接比较的量化证据。

研究问题 研究问题一（RQ1）：大型执行轨迹可视化是否能提高典型程序理解任务的正确率？

研究问题二（RQ2）：在提供相同目标系统的情况下，可视化是否能减少完成理解任务所需的时间？

研究方法 作者设计 8 项面向代表性软件系统的程序理解任务，以 Eclipse 作为对照条件，以 Eclipse 加 Extravis 执行轨迹可视化作为实验条件，比较完成时间和答案正确性。

主要发现 使用轨迹可视化的实验组完成任务所需时间降低 22%，正确性提高 43%，两项差异均达到统计显著。研究证明结构化轨迹的价值可以用客观理解任务而非满意度评价。

贡献与意义 论文为本研究的三条件受控实验提供直接方法学依据：应保持任务和证据可比，测量理解正确率与时间，并把“可视化好看”与“实际理解更准确”区分开。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：34

名称 Debugging Strategies and Tactics in a Multi-Representation Software Environment

作者 Pablo Romero; Benedict du Boulay; Richard Cox; Rudi Lutz; Sallyann Bryant
译者

出处【英国】International Journal of Human-Computer Studies, 2007, 65(12): 992-1009, DOI: 10.1016/j.ijhcs.2007.07.005 页

文献阅读心得（每篇不少于 150 字）：

研究背景 调试环境同时提供代码、图形和运行输出时，用户不仅要理解每种表示，还要在表示之间切换和建立对应关系。表示越多并不自动带来更好表现。

研究问题 研究问题一（RQ1）：程序员如何把高层调试策略与低层界面操作结合，并在多种表示之间组织搜索？

研究问题二（RQ2）：编程经验、图形表示知识和呈现形式如何影响调试准确率及策略适应？

研究方法 研究观察不同经验水平的程序员在多表示软件环境中完成调试任务，结合任务表现和交互过程分析他们查看代码、图形和输出的时间、切换方式及策略变化。

主要发现 成功者更系统地把代码与输出对应起来，并能随呈现条件调整策略；经验较少者更频繁地在表示间切换，形成重复且低效的浏览模式。掌握外部表示规则与编程经验同样重要。

贡献与意义 论文说明本研究的结构图、轨迹和上下文视图必须有明确对应关系，不能只增加表示数量。评价还应观察跨视图跳转、回退和关系核验，而不只记录最终答案。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：35

名称 Are Automated Debugging Techniques Actually Helping Programmers?

作者 Chris Parnin; Alessandro Orso 译者

出处 【加拿大】Proceedings of the 20th International Symposium on Software Testing and Analysis, 2011: 199-209, DOI: 10.1145/2001420.2001445 页

文献阅读心得（每篇不少于 150 字）：

研究背景 自动故障定位常以错误语句排名为目标，并假设程序员只要看到正确位置就能理解和修复问题。然而真实调试还包含依赖导航、状态检查、重跑和根因理解。

研究问题 研究问题一（RQ1）：现有自动调试研究关于程序员如何使用故障排名的假设是否符合实际调试行为？

研究问题二（RQ2）：调试技术应如何评价，才能反映其是否真正帮助开发者理解根因并完成修复？

研究方法 作者分析自动调试和故障定位方法的常见假设，并通过面向开发者的调试任务考察排名信息如何被检查、解释和用于后续导航，进而反思只以错误位置排名评价工具的做法。

主要发现 程序员不会机械地逐项检查排名，也不能仅凭孤立语句理解根因；即使正确语句排名靠前，用户仍需控制流、数据流、状态和上下文证据才能判断。

贡献与意义 论文确立了“自动定位准确”不等于“人能够理解并行动”的方法学原则。本研究因此应直接测量人的事件—机制映射和证据定位，而不能用自动算法指标替代人侧实验。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：36

名称 Characterizing Provenance in Visualization and Data Analysis: An Organizational Framework of Provenance Types and Purposes

作者 Eric D. Ragan; Alex Endert; Jibonananda Sanyal; Jian Chen 译者

出处【美国】IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 31-40, DOI: 10.1109/TVCG.2015.2467551 页

文献阅读心得（每篇不少于 150 字）：

研究背景 可视分析中的溯源信息（provenance）既可指数据来源、系统操作历史，也可指分析者的推理和洞见。概念混用会导致系统记录了大量历史，却不能回答这些记录服务什么任务。

研究问题 研究问题一（RQ1）：可视化与数据分析中的溯源信息可按哪些信息类型和使用目的组织？

研究问题二（RQ2）：不同溯源信息类型应如何影响捕获方式、界面设计和评价方法选择？

研究方法 作者综合多个领域的溯源系统和研究，对记录对象、语义层次、捕获方式及使用目的进行交叉整理，提出组织框架，并讨论类型、目的和评估之间的关系。

主要发现 溯源信息不仅是操作日志，还包括数据、可视状态、分析步骤、推理和洞见等不同层次；回溯、复现、协作、审计和理解等目的需要不同信息，不能由单一历史记录满足。

贡献与意义 论文为本研究区分运行事实、界面推导和人的解释提供概念工具，也说明证据状态必须与具体关系和使用目的绑定，而不能只给整条轨迹一个统一可信度。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：37

名称 Analytic Provenance for Sensemaking: A Research Agenda

作者 Kai Xu; Simon Attfield; T. J. Jankun-Kelly; Ashley Wheat; Phong H. Nguyen; Nallini Selvaraj 译者

出处 【美国】IEEE Computer Graphics and Applications, 2015, 35(3): 56-64, DOI: 10.1109/MCG.2015.50 页

文献阅读心得（每篇不少于 150 字）：

研究背景 意义建构包含信息搜索、假设生成、证据比较和框架修正。若系统只保存点击而不保存分析步骤及其关系，就难以支持反思、恢复中断或追踪不确定性。

研究问题 研究问题一（RQ1）：分析过程中的数据、操作、子任务、假设和推理应以怎样的层级被捕获和呈现？

研究问题二（RQ2）：分析溯源（analytic provenance）如何支持意义建构、协作以及不确定性和信任管理？

研究方法 论文汇总电气电子工程师学会可视化会议（IEEE VIS）相关研讨会和既有研究，围绕溯源信息的捕获、可视化和利用提出研究议程，并用分层模型讨论从低层交互到高层任务语义的推断。

主要发现 低层事件容易自动记录，高层任务和意图难以可靠推断；溯源信息可支持过程反思、探索引导和质量追踪，但推断、隐私、规模化与不确定性传播仍是开放问题。

贡献与意义 论文直接连接溯源信息与数据—框架意义建构，支持本研究把“证据从哪里来、怎样进入结论”作为界面主结构，并要求对自动推断的高层语义显示不确定性。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：38

名称 Capturing and Supporting the Analysis Process

作者 Nazanin Kadivar; Victor Chen; Dustin Dunsmuir; Eric Lee; Cheryl Qian; John Dill; Christopher Shaw; Robert Woodbury 译者

出处 【美国】Proceedings of the IEEE Symposium on Visual Analytics Science and Technology, 2009: 131-138, DOI: 10.1109/VAST.2009.5333020 页

文献阅读心得（每篇不少于 150 字）：

研究背景 分析者会反复调整参数、返回旧状态并探索替代路径，但多数可视分析工具只保存当前结果，不能让人查看、恢复或理解分析过程本身。

研究问题 研究问题一（RQ1）：如何捕获并可视化分析者的交互历史、分支和结果依赖？

研究问题二（RQ2）：可编辑、可回放的分析历史能否支持过程理解、替代路径探索和分析模式复用？

研究方法 作者开发 CzSaw，把用户交互翻译为脚本，生成可编辑的历史视图和结果依赖图，并通过文档集合分析场景展示状态恢复、分支探索和分析流程复用。

主要发现 历史视图使用户能够查看分析进展和替代路径，依赖图则揭示每一步与后续结果之间的逻辑关系；脚本化记录还支持修改和重新执行分析步骤。

贡献与意义 论文提供了“时间历史 + 依赖关系”双重表示的经典先例，与本研究的轨迹和机制并列结构高度对应；它也提示两类视图应联动而非各自展示重复信息。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：39

名称 The Role of Uncertainty, Awareness, and Trust in Visual Analytics

作者 Dominik Sacha; Hansi Senaratne; Bum Chul Kwon; Geoffrey Ellis; Daniel A. Keim 译者

出处【美国】IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 240-249, DOI: 10.1109/TVCG.2015.2467591 页

文献阅读心得（每篇不少于 150 字）：

研究背景 可视分析系统从数据、模型到界面和人的解释都会引入或继承不确定性。若使用者看不到不确定性如何传播，就可能对结果形成过强或过弱的信任。

研究问题 研究问题一（RQ1）：不确定性如何在可视分析的数据处理、模型、可视化和人类推理之间传播？

研究问题二（RQ2）：不确定性意识如何影响证据搜索、验证循环和信任形成？

研究方法 作者整合可视分析、认知和信任研究，提出包含知识生成、探索、验证、不确定性传播与信任构建的概念模型，并通过分析场景说明各环节关系。

主要发现 不确定性会在系统组件中继承和累积，使用者的意识决定其是否继续寻找证据、质疑结果或采取行动；信任应建立在可检查证据和验证循环上，而非结果外观。

贡献与意义 论文为本研究的证据状态提供理论支撑：界面不仅要显示关系，还要让使用者意识到关系的证据强度与未知范围，从而减少由完整结构图诱发的过度确定。

华中科技大学设计学院硕士研究生

文献阅读卡

学号：M202570493

姓名：陈思静

序号：40

名称 Why Authors Don't Visualize Uncertainty

作者 Jessica Hullman 译者

出处【美国】IEEE Transactions on Visualization and Computer Graphics, 2020, 26(1): 130-139, DOI: 10.1109/TVCG.2019.2934287 页

文献阅读心得（每篇不少于 150 字）：

研究背景 尽管不确定性表达方法很多，公开可视化仍经常给出确定性的单一叙事。问题不仅是缺少图形技术，还包括作者对受众、说服力和信息复杂度的假设。

研究问题 研究问题一（RQ1）：经常制作可视化的作者为何承认不确定性重要，却仍选择不直接呈现？

研究问题二（RQ2）：哪些工作实践、态度和传播目标维持了不确定性省略的规范？

研究方法 研究调查 90 名经常为他人制作可视化的作者，并访谈 13 名有影响力的可视化设计者；作者据此归纳实践障碍和信念矛盾，提出不确定性省略的修辞模型。

主要发现 作者普遍认可不确定性的价值，但担心它削弱清晰信息、增加受众负担或破坏叙事。现实中不确定性因此经常被省略，完整而确定的图形外观会掩盖推断自由度和证据边界。

贡献与意义 论文解释了为什么“证据状态应显示”并不会自然转化为界面实践。本研究需要验证三类状态怎样呈现才既不造成认知负担，也不让结构图以确定外观掩盖未知。

1 文献综述

大语言模型智能体（large language model agent，以下简称智能体）已由单轮问答系统发展为能够调用工具、访问外部环境、维护上下文并连续执行任务的复合系统。本研究聚焦工具调用型大语言模型智能体，考察上下文构建、工具调用、状态传递与控制条件等运行环节中的故障诊断。智能体的运行结果由基础模型与提示组织、上下文装配、工具接口、控制循环、状态管理和验证规则共同塑造。本文将这组支撑智能体运行的基础设施称为智能体运行框架（agent harness，以下简称运行框架）。当任务结果出现异常时，开发者需要追踪现象的形成过程、核对支持原因判断的运行记录，并识别现有记录无法回答的问题。

智能体工程（agent engineering）研究已经形成结构化日志、轨迹采集、失败分类、自动归因和运行重放等技术路径。这些技术扩大了可观测范围，也提出了新的交互设计问题：分散事件如何形成连续的执行路径，运行事件如何返回对应的提示、工具、配置或代码位置，系统建立的关系如何说明依据，以及记录不完整时如何表达现有信息的边界。本章据此从系统运行评价、开发者故障分析、证据组织与交互呈现、智能体可观测工具及设计评价方法五个方面梳理相关研究。

1.1 大语言模型智能体故障诊断的研究基础

1.1.1 从模型评价到系统运行评价

早期大语言模型研究多以准确率、胜率或基准任务得分衡量模型能力。在真实产品中，用户获得的结果还受到提示、检索、工具、界面、业务规则和协作流程影响。van der Maden 等人^[1]访谈了 19 名大语言模型产品实践者，其中 17 人遇到了“结果与可行动性鸿沟”：团队能够收集分数与反馈，却难以据此判断应修

改哪个系统环节。Khanna 等人^[2]提出的人工智能行动后复盘方法（After-Action Review for AI, AAR/AI）将总体表现分解到具体决策和内部预测，有助于用户识别已知故障，但实验结果同时显示该方法仍存在误报。这些研究说明，最终得分不能代替对系统运行过程的分析，局部异常也不能直接写成确定根因。

运行框架研究进一步揭示了系统评价的层次性。Chen 等人^[3]将运行框架划分为执行环境与沙箱、工具接口、上下文与记忆、生命周期与编排、可观测、验证评估和治理安全等层次，失败证据可能分散在多个位置。Lin 等人^[4]将系统提示、工具、中间件、技能、子智能体、长期记忆和配置视为可编辑的框架组件，并强调组件变更应当可追踪、可比较和可回滚。Amini 等人^[5]在四种运行时上执行相同的智能体声明，仍观察到准确率、延迟和执行行为的差异。因此，智能体运行问题分析需要以实际执行记录为基础，考察不同机制在具体运行中如何发挥作用。

1.1.2 运行轨迹、问题归因与证据边界

智能体执行轨迹（execution trace）通常包含模型请求与响应、工具调用与返回值、文件或环境变化、智能体间消息、重试和终止等事件。轨迹能够描述“发生了什么”，原因判断还需要说明事件之间的关系以及关系成立的依据。Zhang 等人^[6]将多智能体故障归因分为责任智能体和责任步骤两个维度。在 127 个系统、184 个归因任务中，最佳方法识别责任智能体的准确率为 53.5%，定位具体步骤的准确率仅为 14.2%。自动归因因而适合提供候选调查范围，其结果仍需由开发者结合原始记录进行核查。

运行框架修复工具 HarnessFix^[3]将原始轨迹与框架实现编译为面向运行框架的轨迹中间表示（Harness-aware Trace Intermediate Representation, HTIR），通过数据流、控制流和实现锚点将失败步骤映射到责任层次。智能体框架工程方法 AHE^[4]则利用组件可观测、经验可观测和决策可观测形成自动演化闭环。这些工作证明了轨迹与运行机制之间的关系可以被计算，但面向人的界面还需要解释关

系的来源、前提和适用范围。

本研究中的运行证据包括可返回源记录的事件、事件之间的可复现关系，以及现有记录无法覆盖的信息。证据链以一次实际执行为主线，将上下文片段、模型请求、工具输入输出和状态变化组织为数据流与控制流，并把关键事件连接到提示模板、工具模式、配置规则或实现代码等机制位置。每项信息同时保留来源、处理过程和覆盖范围。

这类组织方式对应三项基本诊断活动：重建执行过程、定位支持判断的运行材料、核验界面关系的生成依据。证据链的作用可通过运行理解、证据定位、判断校准和交互成本进行评价。

1.2 开发者故障分析的信息行为与认知基础

1.2.1 信息搜寻与层级排查

传统调试研究将故障定位与错误修复区分为相互关联但并不等同的任务。Katz 和 Anderson^[7]发现，程序员即使能够修复已指明位置的错误，也未必能根据异常行为顺利找到该位置。开发者需要在代码、运行状态、输出结果和外部文档之间持续移动，以局部线索逐步缩小问题范围。

Lawrance 等人^[8]运用信息觅食理论（information foraging theory）解释程序员的调试行为。开发者会依据错误消息、标识符、文件名和调用关系形成“信息气味”（information scent），并据此选择是否继续追踪当前路径。Ko 等人^[9]对软件维护活动的观察显示，参与者平均约有 35% 的有效时间用于导航、建立关系和收集信息。Rasmussen 和 Jensen^[10]对电子设备排故的研究则发现，熟练人员会在系统、功能单元和部件之间切换层级，同时权衡线索收益、测量成本和认知负担。以上结果表明，运行证据界面需要同时支持问题范围的逐步收缩和不同分析层级之间的连续转换。

1.2.2 问题导向的证据获取与多表征协调

Ko 和 Myers^[11-13]围绕“为什么发生”与“为什么没有发生”问题（why and why-not questions）开展了一系列研究。其为什么线索调试界面（Whyline）允许用户从可见行为提出问题，再查看相关运行事件和代码位置。实验显示，这种以问题组织执行信息的方式能够缩短部分故障定位时间，并提高参与者完成真实调试任务的比例。其设计价值在于建立了问题、运行事件与实现位置之间的可追溯关系。

多表征环境中的关键困难是保持视图之间的语义对应。Romero 等人^[14]研究同时呈现代码、输出和图形表示的调试环境，发现用户表现取决于其能否理解外部表示，并在不同表示之间建立联系。Cornelissen 等人^[15]对执行轨迹可视化开展受控实验，在八项程序理解任务中，轨迹可视化组的用时减少 22%，答案正确性提高 43%。该结果支持结构化轨迹对程序理解的促进作用，也说明表征效果与任务类型、用户经验和视图协调成本有关。

1.2.3 解释框架的形成与更新

故障分析包含从观察现象到形成原因解释的认知转换。Klein 等人^[16-18]将意义建构（sensemaking）描述为分析者利用既有经验组织信息，并在新线索出现后持续调整解释的过程；其数据—框架理论（Data-Frame theory）进一步指出，数据既被当前框架选择和解释，也可能扩展、质疑或替换该框架。在智能体运行问题中，解释框架可能表现为对工具调用、上下文遗漏、状态覆盖或停止条件的原因假设；后续证据可能支持该假设，也可能要求开发者比较其他解释。

Pirolli 和 Card^[19]通过认知任务分析，将复杂分析划分为信息觅食循环和意义建构循环：前者负责搜索、筛选和整理材料，后者负责建立证据结构、生成假设并形成结论。Kadivar 等人^[20]开发的 CzSaw 可视分析系统记录分析历史、依赖关系和分支，允许分析者返回旧状态并探索替代路径。Xu 等人^[21]区分低层事件、

分析动作、子任务和总体任务，并指出语义层级越高，系统越难仅依据交互日志可靠推断人的意图。运行问题分析工具因而需要区分机器可记录的操作与分析者主动形成的判断。

Starr 和 Storey^[22]根据 27 名开发者的访谈，将故障排查概括为困惑、试探、观察和恢复等过程，并指出困惑会占用注意与工作记忆。界面应帮助开发者保留当前问题、候选原因和已核对证据，使其在跨视图导航或分析中断后能够恢复判断上下文。

1.2.4 心智模型、解释与适当依赖

可解释人工智能（explainable artificial intelligence, XAI）研究区分了系统生成解释与用户形成正确理解。Miller^[23]综合哲学、认知心理学和社会心理学研究，指出人类解释具有选择性、对比性和社会性，单向输出特征重要度并不必然改善理解。Abdul 等人^[24]对 289 篇论文及其引用网络的分析也表明，可解释、可问责与可理解系统需要算法研究与人机交互研究共同推进。

解释工具还会改变用户对系统能力的判断。Kaur 等人^[25]通过 11 名数据科学家的情境调查和 197 人问卷发现，专业用户同样可能误用或过度信任解释工具，并且难以准确说明部分可视化的含义。Sieker 等人^[26]的实验显示，解释提高了用户对系统能力的主观评价，却没有同步提高其识别系统局限的能力。Buçinca 等人^[27]研究的认知强制交互通过延迟建议、要求独立判断等方式降低了用户对错误人工智能建议的过度依赖，同时增加了交互成本。证据呈现的评价因此需要联合考察答案正确性、证据使用和信心水平，以判断使用者是否形成了适当依赖（appropriate reliance）。

多智能体界面中的心智模型还受到角色数量、交接关系和意见一致性的影响。Pareek 等人^[28]使用不同界面方案作为研究探针，发现用户会依据代理角色、任务转移和相互批评等线索判断系统可靠性。Kumar 等人^[29]对真实开发者—智能体协作的研究显示，持续检查、分阶段验证和主动干预仍是有效协作的重要组

成部分。这些结果说明，结构线索和社会线索能够帮助理解，也可能在缺少实质证据时产生不恰当的可靠性感知。

1.3 运行证据的组织与交互呈现

1.3.1 多层级运行信息的结构化表征

智能体运行信息具有时间、组件、数据和控制等多个组织维度。时间线适合呈现事件顺序，调用树适合展示层级关系，数据流图适合追踪输入输出的传递，状态视图则用于比较运行前后的变化。单一表征难以覆盖全部分析任务，界面需要明确各种关系的语义，并在概览与原始记录之间保持稳定的定位方式。

Vicente 和 Rasmussen^[30]提出的生态界面设计（ecological interface design）强调，界面应呈现工作领域中具有约束作用的对象与关系，使操作者能够在非预设情境下理解系统状态并采取行动。应用于智能体运行分析，工作领域中的关键对象包括任务、模型请求、工具、上下文片段、状态对象和系统配置；关键关系则包括调用、传递、变换、覆盖和终止。该理论为运行证据的信息分层和关系表达提供了设计依据。

结构化表征仍需控制聚合带来的信息损失。轨迹概览能够缩小搜索范围，但压缩过程可能省略关键参数、异常返回值或消息边界。设计上需要保留由概览返回原始事件的路径，并说明聚合规则、覆盖范围和可能遗漏的信息。

1.3.2 证据关联与分析溯源

分布式追踪研究为跨步骤证据关联提供了成熟的数据结构。Fonseca 等人^[31]提出的跨层追踪系统 X-Trace 通过显式传播任务标识，将跨应用、协议和管理域的事件重建为任务树。Sigelman 等人^[32]提出的分布式追踪基础设施 Dapper 使用追踪标识（trace ID）和跨度标识（span ID）表示一次请求的分布式路径。Mace 等人^[33]提出的动态追踪方法 Pivot Tracing 依据先发生关系连接跨组件事件，

并支持在运行过程中查询新的关系。这些工作说明，可靠关联依赖稳定标识、上下文传播和明确规则；采样、未插桩组件或标识传播中断都会留下信息空白。

可视分析领域将分析溯源（analytic provenance）扩展到人的分析过程。Ragan 等人^[34]将溯源信息区分为数据、可视化、交互、洞见和理由等类型，并强调记录对象应与使用目的相匹配。Xu 等人^[21]指出，从低层交互自动推断高层任务与意图存在语义不确定性，分析记录应支持回看、恢复、协作和质量检查。Kadivar 等人^[20]通过可编辑历史和结果依赖图呈现分析路径、分支与中间结果之间的联系。

面向智能体运行问题，证据溯源包含两个方向：其一，从界面中的关系返回原始事件与实现位置，确认信息来源和生成条件；其二，从某一事件沿数据流或控制流追踪后续传播，理解该事件在完整执行过程中的位置。交互界面需要保持两个方向的一致性，并记录系统生成关系所使用的输入和规则。

1.3.3 证据状态、不确定性与信息缺口

运行证据的状态由信息的获得方式决定。调用参数、返回值和状态写入属于实际记录；依据追踪标识、字段映射或程序规则建立的关系属于规则推导；未采集字段、不可见工具状态和无法重建的跨系统关系属于目前未知。三类状态分别对应直接查看、核验规则和保留判断等分析活动，需要清晰的文字与视觉区分。

Sacha 等人^[35]将不确定性视为贯穿数据采集、模型处理、视觉表达和人的解释全过程的属性。Hullman^[36]调查了 90 名可视化作者并访谈 13 名设计者，发现作者虽然承认不确定性信息具有价值，却常因担心界面复杂、说服力下降和读者负担而省略此类信息。Dritsa 和 Houben^[37]对设计研究者使用数据可视化的研究进一步表明，不确定性可能来自数据质量、分析参数和使用者背景，设计需要帮助使用者辨认来源，并判断其对结论的影响。针对图边不确定性的受控实验还发现，视觉变量的效果会受主属性编码、可辨识度和任务类型共同影响^[38]，说明证据状态的表达需要先经过任务导向的小规模比较，不宜直接假定某一视觉编码普遍有效。

Ehsan 等人^[39]提出的技术接缝显化解释方法（Seamful XAI）关注数据、模型和基础设施边界对使用者行动能力的影响。该研究表明，呈现系统限制与技术接缝有助于使用者识别可操作范围。在运行问题分析中，“目前未知”可以直接连接受影响的问题和补充检查，使开发者决定是否进行重放、增加埋点、对照测试或环境核查。

证据状态描述来源与处理方式，不构成简单的高、中、低可信度排序。实际记录可能受到采样和埋点范围限制，规则推导在前提满足时可以稳定复现，目前未知则表明现有材料无法支持确定结论。状态设计需要让使用者看见信息来自何处、经过何种处理以及覆盖到何种范围。

1.4 大语言模型智能体可观测与交互调试研究

1.4.1 运行轨迹采集与系统可观测

智能体可观测性（observability）研究首先需要确定记录范围。结构化日志框架 AgentTrace^[40]将日志分为操作、认知和上下文三个平面，用以组织模型交互、环境动作和上下文变化。系统级可观测工具 AgentSight^[41]同时捕捉模型侧语义信息与操作系统层事件，扩展了对外部进程行为的覆盖。开放智能体规范（Open Agent Specification）^[5]则尝试以统一声明支持不同智能体框架之间的运行比较。这些工作提高了运行材料的可获得性和可计算性，但其研究重点主要是记录范围、数据结构和系统开销。

分层可观测数据并不会自动形成面向开发者的解释。内核事件属于直接观测，高层意图及两者之间的对应关系可能依赖规则或模型推断；相同事件在任务、工具、上下文和状态视图中的含义也不同。可观测基础设施需要通过信息架构和交互操作转化为可核对的分析材料。

1.4.2 轨迹浏览、关系分析与运行干预

现有智能体调试工具可按主要交互方式分为轨迹浏览、关系分析和运行干预三类。行为分析工具 AgentLens^[42]使用层级时间视图呈现多智能体行为的演变，并支持从整体模式进入具体行为。长任务观察工具 AgentGUI^[43]同时展示任务、轨迹、产物和成本；在 8 人实验中，参与者定位关键轨迹信息的平均用时由 145 秒降至 90 秒，答案正确率由 80% 提高到 93%。这类工具证明了概览与细节联动的价值，但候选原因和自动建议仍需标明其推断性质。

关系分析工具使用图结构呈现跨步骤依赖。智能体图分析平台 Agent-Graph^[44]将智能体、任务、工具、输入输出和人员组织成知识图谱，每个图元素均可返回对应轨迹片段。轨迹图系统 Graph of Trace^[45]将科学研究智能体的执行事件组织为持续更新的有向图，支持领域专家检查 workflows 和中间步骤。图结构能够表达长链依赖，但节点、边和聚合规则需要与用户任务相匹配，否则视觉连线容易被理解为已经确认的因果关系。

运行干预工具将观察与验证操作结合。多智能体交互调试工具 AGDebugger^[46]支持暂停、重置和修改历史消息。其 14 人实验显示，交互调试有助于定位部分问题，但参与者在选择干预位置和修改方式时仍经常失败。软件开发智能体调试工具 AgentStepper^[47]引入断点、单步执行、中间状态检查和现场编辑；在 12 人实验中，参与者识别智能体实现缺陷的成功率由 17% 提高到 60%，一般轨迹理解成绩则仅由 64% 提高到 67%。用户能够操作轨迹，并不意味着已经形成正确的问题解释。

无代码智能体开发工具 AutoGen Studio^[48]提供 workflow 配置、运行试验和消息查看能力，数据应用调试工具 LADYBUG^[49]支持逐步检查输入输出、局部干预和受影响步骤的重新计算。此类工具降低了比较替代路径的成本，也使观察、判断与系统修改紧密相连。证据尚不充分时的修改可能产生新的混杂因素，因此界面需要保留干预前的判断、所依据的证据和预期结果。

表 1.1 代表性智能体运行分析工具的关联与状态支持比较

工具	分析入口	跨步骤关系	实现位置关联	证据状态表达	评价重点
AG De-bugger ^[46]	代理消息	消息顺序与编辑分支	可定位消息与干预点	原始消息可查；推导与未知区分有限	定位策略和干预
Agent-GUI ^[43]	任务与轨迹	时间和产物关联	任务、轨迹与产物联动	自动判断与记录来源区分有限	查找时间、正确率和负担
Agent-Stepper ^[47]	动作与代码变化	模型-程序-工具会话链	断点与代码现场关联	保留原始状态；未知信息提示有限	轨迹理解和缺陷识别
Agent-Lens ^[42]	层级时间视图	行为演化与下钻	以行为层级为主要定位方式	未系统区分记录、推导和未知	行为模式分析
Agent-Graph ^[44]	知识图谱	任务、工具与输入输出关系	图元素可返回轨迹片段	关系生成依据与未知范围表达有限	系统能力和案例展示
Graph of Trace ^[45]	workflow图	实时事件与中间产物	对应 workflow阶段与产物	支持过程检查；状态语义有限	workflow可理解性

1.4.3 自动分析结果的人因局限

自动失败归因和修复研究将轨迹处理为责任步骤、关联关系或修复建议。此类结果依赖日志覆盖、分类规则、模型能力和评测标签。Parnin 和 Orso^[50]研究自动故障定位工具后指出，算法将错误语句排在前列，并不意味着程序员能够理解根因；真实调试还需要沿数据依赖和控制依赖导航、检查状态并通过重运行验证判断。

面向开发者的运行分析工具需要为自动生成内容保留原始记录入口、计算规则和适用前提，并在必需信号缺失时明确显示未知状态。评价对象也需要覆盖开发者的过程理解、证据定位和结论校准。

1.5 证据呈现的设计研究与评价方法

1.5.1 形成性研究与表征设计

证据呈现设计需要建立在开发者实际分析活动的观察之上。关键事件访谈、案例回放、出声思考和认知任务分析可用于识别线索来源、搜索路径、判断转折和信息缺口。真实案例能够保留工作情境、噪声和不完整数据，统一案例则便于比较不同参与者的证据选择和判断过程。两类材料结合可以兼顾生态效度与任务可比性。

设计探索阶段应将信息层级、关系语义和交互入口转化为可比较的表征方案。Pareek 等人^[28]以不同多智能体界面作为研究探针，分析界面结构如何影响用户对角色、任务转移和可靠性的理解。这种研究方法适合在完整原型实现之前检验表征假设，并识别视觉结构可能引发的判断偏差。

形成性发现、设计要求与界面机制之间需要建立可追踪关系。每项主要功能应说明其回应的分析困难，并在后续评价中对应至少一项可观察结果。该做法能够避免将一般性的界面偏好直接解释为设计有效性。

1.5.2 证据呈现的评价维度

综合调试研究、可视分析和人与人工智能交互（Human-AI Interaction）研究，证据呈现可从四个维度评价。第一，运行理解，包括执行顺序、数据来源、分支条件和状态变化的正确性。第二，证据定位，包括关键事件、字段和实现位置的查找准确率。第三，判断校准，包括信息充分与不充分情境下的答案、信心和肯定判断比例。第四，使用成本，包括任务时间、交互操作量、主观负荷和可用性。

这些指标分别对应理解、定位、判断和成本，能够呈现界面设计对诊断活动的具体影响。信心水平与答案正确性需要联合分析，以区分合理信心、谨慎判断和高信心错误。

1.5.3 实验条件与分析方法

证据呈现研究需要区分底层运行材料、事件关联和证据状态的作用。实验可设置三个递进条件：原始时间轨迹与静态结构视图（A）；在 A 基础上连接数据流、控制流和实现位置的关联视图（B）；在 B 基础上标明实际记录、规则推导和目前未知的状态视图（C）。预先设定的 B-A 对比用于评价轨迹—机制关联，C-B 对比用于评价证据状态标注。三种条件使用相同的底层记录，使比较集中于信息组织与状态表达。

对于由多个参与者和多个案例构成的实验，线性混合效应模型或广义线性混合效应模型能够同时处理个体差异与案例差异。质性访谈、出声思考和交互日志则用于解释定量结果，区分界面未提供关键线索、参与者未发现线索和参与者发现线索后仍作出错误推断等情况。

1.6 国内外研究进展与本研究定位

国内研究已围绕智能时代的人机交互、人机协同、人工智能决策可解释性及其评价方法展开。范俊君等人^[51]讨论了智能时代人机交互中的心理模型、交互

原则与评价框架。范向民等人^[52]梳理了人机交互与人工智能从交替发展到协同共进的关系。孔祥维等人^[53]和李瑶等人^[54]分别从人工智能决策与评价方法角度总结了可解释性研究。邢颖^[55]则对可解释人工智能在软件工程中的技术方法进行了综述。这些研究为以人为中心的解释设计和评价提供了理论基础，直接面向智能体运行证据的人因研究仍相对有限。

国外研究已经形成从程序调试、分布式追踪、可视分析到智能体交互工具的连续脉络。信息觅食和问题导向调试研究解释了开发者如何寻找线索；分布式追踪和分析溯源研究建立了事件关联与分析历史的数据基础；不确定性、解释和适当依赖研究说明了呈现方式对判断与信心的影响；近期智能体工具则提供了轨迹采集、图结构分析、运行重放和交互干预能力。

现有研究为本课题留下三个相互连接的研究空间。第一，智能体工具已经能够采集长轨迹，但运行事件与数据传递、控制过程和实现位置之间的对应关系仍缺少面向开发者任务的组织。第二，图结构能够压缩复杂执行过程，图中连线的来源、生成规则与未知范围需要清晰表达。第三，现有评价多集中于系统能力、算法准确率、任务时间或主观可用性，对运行理解、证据定位、信息不足时的判断和信心校准缺少联合考察。

本研究以工具调用型大语言模型智能体故障诊断为应用情境，从开发者实际诊断活动出发，研究跨步骤、跨组件证据关联的需求与困难；在此基础上设计连接运行事件、数据流、控制流和实现位置的证据链界面，并探索证据状态的表达方式；最后通过分层对照实验评价这些设计对理解、定位、判断校准和使用成本的影响。

1.7 本章小结

相关研究为智能体运行证据链设计提供了三方面基础。程序调试与意义建构研究解释了开发者如何搜索线索并重建执行过程；分布式追踪、分析溯源和不

确定性研究提供了组织来源、关系与信息边界的方法；智能体可观测和交互调试工具证明了长轨迹浏览、关系分析、运行回放和实现定位的技术可行性。

本研究把证据呈现作为连接可观测数据与开发者诊断活动的交互层。形成性研究用于识别关联需求与理解困难，表征设计用于形成轨迹—机制关联和证据状态的界面语义，对照实验用于评价设计对运行理解、证据定位、判断校准与认知负荷的影响。

2 选题报告

2.1 研究背景

大语言模型智能体通过模型推理、上下文构建、工具调用、状态维护和流程控制完成多步骤任务。一次执行往往跨越提示模板、模型接口、工具适配器、中间件、状态存储和终止规则。任务结果出现异常时，开发者需要回答三个相互关联的问题：执行过程中发生了什么，相关信息经过哪些步骤与组件，异常最终对应哪一处运行机制。

智能体平台通常提供按时间排列的运行轨迹，以及描述代理、工具和任务关系的静态结构视图。时间轨迹便于查看单个事件，静态视图便于认识系统组成；故障定位还需要把某项输入、工具返回值或状态变化沿执行过程连续追踪，并回到提示模板、工具定义、控制规则或实现代码中的具体位置。随着轨迹增长，事件之间的数据传递与控制转移难以从相邻日志中直接看出，结构图中的连线也可能被理解为已经发生的运行关系。

运行材料还具有不同的证据状态。模型请求、工具返回和状态写入属于实际记录；依据追踪标识、程序规则或字段映射建立的关系属于规则推导；未采集、无法访问或无法由现有记录确定的信息属于目前未知。三类状态如果采用相同的视觉形式，使用者容易把系统补出的关系视为直接观测，也可能在关键信息缺失时作出肯定判断。

本研究以工具调用型大语言模型智能体为对象，聚焦上下文构建、工具调用、状态传递和控制/停止条件中的运行故障。研究将证据链定义为由运行事件、数据流、控制流、实现锚点和证据状态共同构成的可追溯结构，题目确定为“大语言模型智能体故障诊断的证据链交互设计研究”。

2.2 研究目的与意义

2.2.1 研究目的

本研究旨在理解开发者诊断智能体运行故障时对跨步骤、跨组件证据关联的需求，设计能够连接运行轨迹与运行机制的证据链界面，并评价证据关联与状态标注对运行理解、证据定位和结论校准的影响。研究形成三个层次的成果：故障诊断任务与证据需求模型；轨迹—机制关联及证据状态的交互设计方案；基于受控案例的实证评价结果。

2.2.2 理论意义

研究将程序调试中的信息搜寻、多表征协调和意义建构研究拓展到大语言模型智能体场景，描述开发者如何在概率性执行、跨组件传播和部分可见的运行环境中重建过程。研究进一步区分运行事件、关系推导和未知信息，建立界面设计、理解与定位活动、诊断判断之间的解释框架，为智能体可观测性的人因研究提供可操作的概念与测量方法。

2.2.3 实践意义

研究成果可用于智能体开发平台、运行监控系统和调试工具的信息架构设计。相关界面能够帮助开发者沿数据流和控制流追踪异常，快速进入对应的提示、工具、配置或代码位置，并识别结论所依据的是原始记录、规则推导还是不完整信息，从而支持故障定位、复盘沟通和协作检查。

2.3 国内外研究现状

程序调试研究表明，故障定位包含线索搜寻、跨表征导航和因果解释。Ko 和 Myers^[11-13]以“为什么发生”和“为什么没有发生”组织运行信息，将可见结

果连接到执行事件和代码位置。Lawrance 等人^[8]运用信息觅食理论解释开发者如何根据错误消息、标识符和调用关系选择搜索路径。Cornelissen 等人^[15]的受控实验则显示,结构化轨迹能够改善部分程序理解任务,但效果受到任务类型、用户体验与视图协调成本影响。

分布式追踪与分析溯源研究为跨步骤证据关联提供了技术基础。X-Trace^[31]、Dapper^[32]和 Pivot Tracing^[33]利用追踪标识、跨度关系和先发生关系重建跨组件执行路径。Ragan 等人^[34]将分析溯源划分为数据、可视化、交互、洞见和理由等类型,说明不同分析任务需要记录不同层次的来源信息。上述研究共同指出,事件关联依赖明确的标识传播和生成规则,采样、缺少埋点或跨系统边界会形成无法直接填补的信息空白。

智能体可观测与交互调试工具已经支持轨迹采集、图结构分析和运行干预。AgentLens^[42]使用层级时间视图呈现行为演变, AgentGraph^[44]和 Graph of Trace^[45]将任务、工具与输入输出组织为图结构, AGDebugger^[46]与 AgentStepper^[47]支持暂停、单步和修改运行状态。HarnessFix^[3]进一步使用数据流、控制流和实现锚点连接轨迹与运行框架实现。这些工作提供了构建证据链所需的数据结构和交互基础,也表明轨迹浏览、关系理解与故障定位是不同层次的用户任务。

不确定性可视化和可解释人工智能研究关注信息来源、推导过程与判断信心。Sacha 等人^[35]将不确定性视为贯穿数据采集、处理、呈现和解释的属性。Guo 等人^[38]发现,图边不确定性的视觉编码效果与主属性编码和任务类型有关。Kaur 等人^[25]和 Sieker 等人^[26]的研究表明,专业用户也可能误解解释界面,并在理解没有改善时提高对系统的主观评价。因此,智能体故障诊断界面的评价需要同时考察答案正确性、证据选择、判断信心和任务负担。

现有研究已经形成运行轨迹、结构关系和交互调试等基础能力。本研究首先考察开发者在真实诊断活动中如何寻找、连接和核验运行证据,再根据形成性研究结果设计跨步骤关联与证据状态的呈现方式,最后在底层运行材料一致的条件下评价这些设计对故障诊断的影响。

2.4 研究问题

研究问题一（RQ1）：开发者在理解智能体运行过程与定位故障时，如何跨越执行步骤和系统组件搜寻、关联与核验运行证据？这一过程中存在哪些主要困难和信息需求？

研究问题二（RQ2）：运行事件与数据传递、控制过程及实现机制之间的可追溯关联，如何影响开发者对执行过程的理解和故障定位表现？

研究问题三（RQ3）：对不同证据状态进行显式区分，如何影响开发者在运行信息不完整情境下的判断、信心校准与认知负荷？

2.5 核心概念与研究范围

2.5.1 轨迹—机制关联

轨迹—机制关联连接一次实际执行与运行框架中的对应机制，包括三类关系：

- 1) **数据流**：上下文片段、参数、工具结果和状态值在步骤之间的来源、变换与去向；
- 2) **控制流**：分支选择、重试、回退、终止与下一步调度之间的执行关系；
- 3) **实现锚点**：运行事件对应的提示模板、工具模式、配置项、中间件规则、状态更新逻辑或代码位置。

每条关联均保留起点、终点、生成规则和原始记录入口，使使用者能够在图形关系、事件详情和实现材料之间双向定位。

2.5.2 证据状态

证据状态描述界面信息的获得方式和可核验范围：

- 1) **实际记录**：运行时直接采集且能够返回源记录的事件或字段；

2) **规则推导**：根据确定的追踪标识、字段映射或程序规则计算得到的关系，并显示所用规则与前提；

3) **目前未知**：现有材料无法确定的信息，包括未采集字段、不可见组件状态和无法重建的跨系统关系。

证据状态表示信息来源，不作为简单的可信度等级。实际记录可能存在采样遗漏，规则推导在前提满足时可以稳定复现，目前未知则提示使用者保留判断或补充检查。

2.6 研究内容与技术路线

研究依次开展形成性研究、表征设计、原型实现和正式实验。形成性研究回答 RQ1，并决定正式实验中的任务类型与界面需求；正式实验中的 A-B 比较回答 RQ2，B-C 比较回答 RQ3。

2.6.1 形成性研究

计划招募 10–14 名参与者，纳入条件为至少参与过一个工具调用型智能体项目，或具有一年以上复杂软件系统调试经验。研究采用关键事件访谈、脱敏案例回放和统一案例中的出声思考，覆盖执行过程复原、关键证据定位、跨组件关系解释和证据充分性判断等任务。

研究记录参与者的起始线索、查看顺序、跨视图跳转、关系解释、证据引用和判断依据。资料采用框架分析法，两名研究者对不少于 25% 的材料独立编码，形成稳定的编码手册后完成全量分析。输出包括任务模型、证据关联需求、常见误解、状态区分需求，以及“研究发现—设计要求—界面功能—评价指标”对应表。

2.6.2 表征设计研究

根据形成性研究结果制作 3–5 种低保真方案，比较时间轴、流程图、并列视图和按问题展开的关联视图。计划邀请 6–8 名开发者开展任务式比较走查，并邀请 3–5 名人机交互或开发者工具专家进行启发式评审。

表征评价关注六项内容：跨步骤定位是否连续，数据流和控制流是否容易区分，实现锚点是否明确，关系生成依据是否可查看，三类证据状态是否易辨认，以及界面密度与操作成本是否可接受。该阶段形成原型信息架构、视觉语义和交互规格。

2.6.3 交互原型

原型命名为“智能体运行证据链工作台”，由轨迹视图、机制视图和证据详情面板组成，提供以下功能：

- 1) 从异常输出、工具结果或状态变化进入相关轨迹片段；
- 2) 沿数据流查看值的来源、变换和传播路径；
- 3) 沿控制流查看分支、重试、回退和终止过程；
- 4) 从运行事件跳转到提示、工具定义、配置、规则或代码锚点；
- 5) 查看关系的生成规则、输入字段和适用前提；
- 6) 通过文字标签与视觉编码区分实际记录、规则推导和目前未知；
- 7) 记录参与者选取的证据、最终判断与信心分配。

2.6.4 正式实验

计划招募 36–48 名具有软件开发经验的参与者，记录其智能体开发经历、调试年限和基础能力。实验优先采用被试内设计，每名参与者先完成练习任务，再完成 6 个正式案例。条件顺序和案例—条件映射采用拉丁方与分块随机化平衡。样本量根据预先设定的最小实际重要效应进行模拟功效分析。

表 2.1 实验条件及其对应研究问题

条件	界面内容	研究作用
A 基础视图	原始时间轨迹、事件详情、搜索筛选，以及独立的静态组件结构图	提供常用分析材料与操作基线
B 关联视图	A 的全部内容，加上数据流、控制流和实现锚点；关系可返回原始事件与生成规则	与 A 比较，回答 RQ2
C 状态视图	B 的全部内容，加上“实际记录”“规则推导”“目前未知”的文字与视觉标记	与 B 比较，回答 RQ3

三种条件使用相同的底层运行记录、静态结构信息和案例问题。B 与 C 中的关系由同一套确定性规则生成，C 只增加状态说明。实验任务和主要指标在数据收集前预注册，A-B 与 B-C 为预设比较。

2.7 研究材料与案例构建

形成性研究可使用脱敏自然案例与受控案例。正式实验使用能够固定运行材料和判断依据的受控案例，覆盖上下文构建、工具调用、状态传递和控制/停止条件。每个案例包含执行过程理解题、证据定位题和故障判断题，并根据形成性研究识别出的典型任务确定信息规模与关键关系。

案例采用“固定任务—受控注入—轨迹冻结—反事实验证”的方式构建。研究冻结模型版本、采样设置、工具输出和环境快照，注入指定运行故障后保存三种条件共同使用的记录，再通过移除故障或替换正确状态进行反事实重放。两名智能体系统专家独立核对事件关系、实现锚点和答案依据，由第三位专家处理分歧。

为评价证据状态，部分任务提供足以作出判断的记录，部分任务保留自然产

生或受控移除的关键观测缺口。缺口类型由形成性研究中的实际困难确定，正确回答为“现有证据无法确定”并说明缺少的信息。充分与不充分案例在故障类型、轨迹长度和任务难度上配对，所有条件接收完全相同的底层记录。

2.8 评价指标与分析方法

表 2.2 研究问题、评价指标与操作化方式

对应问题	指标	操作化方式
RQ2	运行理解正确性	对执行顺序、数据来源、分支条件和状态变化问题的答案得分
RQ2	证据定位准确率	所选事件、字段与实现锚点相对于专家答案的准确率、召回率和综合得分
RQ2	定位时间	从呈现案例到首次提交正确证据集合的时间
RQ3	无依据肯定判断率	在关键信息不足任务中选择确定原因或确定关系的比例
RQ3	结论校准	对各答案分配概率，使用 Brier 分数及置信—正确性关系评价
RQ3	任务负担	NASA-TLX、操作数量和主观可用性；同时记录状态标记的理解正确率

运行理解与证据定位得分由案例答案和操作日志共同计算。需要人工评分的开放解释由不知道界面条件的评分者依据手册评分，不少于 25% 的样本进行双人独立评分，并根据变量类型报告组内相关系数或加权 Kappa。

定量分析根据因变量类型采用线性或广义线性混合效应模型，将参与者和案例设为交叉随机效应，在数据支持时加入界面条件的参与者随机斜率；固定效

应包括界面条件、经验水平、任务顺序和案例复杂度。RQ2 重点报告 B-A 对比，RQ3 重点报告 C-B 对比，并报告效应量与置信区间。任务负担采用预注册的可接受范围进行解释，使结论同时反映效果与使用成本。访谈和出声思考用于说明界面如何影响关系理解与证据判断。

2.9 研究概念框架

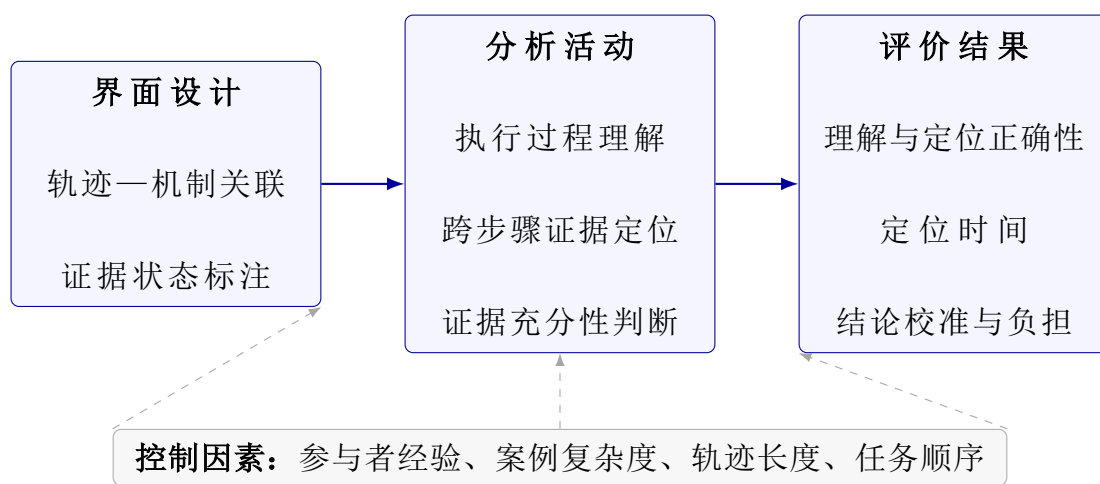


图 2.1 证据链界面影响智能体故障诊断的概念框架

研究路线由任务发现、设计形成和效果评价构成。RQ1 的形成性研究识别开发者实际遇到的关联需求与误解；表征研究将需求转化为可辨认、可追溯的界面语义；正式实验在运行记录一致的条件下依次评价轨迹—机制关联和证据状态标注。

2.10 可行性、风险与伦理

2.10.1 研究可行性

研究可复用现有智能体运行框架、轨迹记录和测试任务。实现工作集中于事件标准化、关系计算、实现锚点映射和界面联动，无需训练新的基础模型。案例范围集中于四类可注入、可回放的运行故障，适合在硕士研究周期内完成。

2.10.2 研究风险与控制

参与者经验差异通过基础能力测量、条件平衡和统计控制处理。案例关系由确定性规则生成并经专家核对，降低自动生成关系错误对实验结果的影响。模型输出波动通过冻结轨迹和反事实重放控制。多视图界面的学习成本通过练习任务、顺序平衡和任务负担测量处理。正式招募不足时，依据功效模拟调整每名参与者完成的案例数量，同时保留三种条件和关键案例类型的覆盖。

2.10.3 研究伦理与数据管理

研究将在获得伦理审查批准后招募参与者。知情同意书说明研究目的、屏幕/语音/交互日志的记录范围、退出权利和补偿安排。自然案例移除代码、密钥、客户数据和组织标识；研究数据加密存储，限定访问人员，并说明保留期限与删除方式。未经充分脱敏的材料不上传至第三方生成式人工智能服务。

2.11 进度安排

表 2.3 研究进度计划

阶段	时间	主要任务与成果
研究准备与案例构建	2026.09–2026.10	补充文献，提交伦理材料，建立关系规则、实现锚点与案例答案手册
形成性研究	2026.11–2027.01	开展案例回放、统一任务和访谈，形成任务模型与证据关联需求
表征探索与原型设计	2027.02–2027.06	比较关联与状态表征，完成原型、专家评审和预实验
正式实验	2027.07–2027.10	完成功效模拟、预注册、正式实验和访谈资料收集
综合分析与论文初稿	2027.11–2028.02	完成统计与质性分析、设计原则归纳和论文初稿
论文修改与答辩	2028.03–2028.05	完成论文修改、送审、答辩准备和研究材料归档

2.12 预期成果与创新点

预期成果包括：智能体故障诊断任务与证据关联需求模型；轨迹—机制关联和证据状态的交互设计原则；智能体运行证据链工作台原型；关于两类界面设计对理解、定位和结论校准影响的实验结果；配套的受控案例与评价材料。

研究创新点包括：

- 1) **研究对象**：以运行框架的执行过程为分析单位，研究模型、工具、状态和控制机制共同作用下的故障诊断；
- 2) **交互设计**：将分散事件组织为数据流、控制流和实现锚点，并保持图形关系、原始记录与实现材料之间的双向追溯；
- 3) **证据表达**：以“实际记录—规则推导—目前未知”描述信息来源与边界，支持使用者判断现有材料能够回答到何种程度；
- 4) **实证评价**：在底层运行记录相同的条件下，通过 A-B 和 B-C 两个预设比较分别评价关系呈现与状态标注的作用。

参考文献

- [1] Van der Maden W, Sadek M, Xiao Z, et al. Results-Actionability Gap: Understanding How Practitioners Evaluate LLM Products in the Wild[C]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026: 1-17.
- [2] Khanna R, Dodge J, Anderson A, et al. Finding AI's Faults with AAR/AI: An Empirical Study[J/OL]. ACM Transactions on Interactive Intelligent Systems, 2022, 12(1): 1:1-1:33. DOI: 10.1145/3487065.
- [3] Chen M, Wang J, Liu Z, et al. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2606.06324>. arXiv: 2606.06324 [cs.SE].
- [4] Lin J, Liu S, Pan C, et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2604.25850>. arXiv: 2604.25850.
- [5] Amini S, Benajiba Y, Bernardis C, et al. Open Agent Specification: Enabling

- Cross-Framework Comparison of AI Agents[C]//Proceedings of the ACM Conference on AI and Agentic Systems. 2026: 404-418.
- [6] Zhang S, Yin M, Zhang J, et al. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems[C]//Proceedings of Machine Learning Research: vol. 267. 2025: 76583-76599.
- [7] Katz I R, Anderson J R. Debugging: An Analysis of Bug-Location Strategies[J]. Human-Computer Interaction, 1987-1988, 3(4): 351-399.
- [8] Lawrance J, Bogart C, Burnett M, et al. How Programmers Debug, Revisited: An Information Foraging Theory Perspective[J]. IEEE Transactions on Software Engineering, 2013, 39(2): 197-215.
- [9] Ko A J, Myers B A, Coblenz M J, et al. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks[J]. IEEE Transactions on Software Engineering, 2006, 32(12): 971-987.
- [10] Rasmussen J, Jensen A. Mental Procedures in Real-Life Tasks: A Case Study of Electronic Trouble Shooting[J]. Ergonomics, 1974, 17(3): 293-307.
- [11] Ko A J, Myers B A. Debugging Reinvented: Asking and Answering Why and Why Not Questions about Program Behavior[C]//Proceedings of the 30th Inter-

- national Conference on Software Engineering. 2008: 301-310.
- [12] Ko A J, Myers B A. Designing the Whyline: A Debugging Interface for Asking Questions about Program Behavior[C]//Proceedings of the 2004 ACM Conference on Human Factors in Computing Systems. 2004: 151-158.
- [13] Ko A J, Myers B A. Finding Causes of Program Output with the Java Whyline [C]//Proceedings of the 27th ACM Conference on Human Factors in Computing Systems. 2009: 1569-1578.
- [14] Romero P, du Boulay B, Cox R, et al. Debugging Strategies and Tactics in a Multi-Representation Software Environment[J]. International Journal of Human-Computer Studies, 2007, 65(12): 992-1009.
- [15] Cornelissen B, Zaidman A, van Deursen A. A Controlled Experiment for Program Comprehension through Trace Visualization[J]. IEEE Transactions on Software Engineering, 2011, 37(3): 341-355.
- [16] Klein G, Moon B, Hoffman R R. Making Sense of Sensemaking 1: Alternative Perspectives[J]. IEEE Intelligent Systems, 2006, 21(4): 70-73.
- [17] Klein G, Moon B, Hoffman R R. Making Sense of Sensemaking 2: A Macrocognitive Model[J]. IEEE Intelligent Systems, 2006, 21(5): 88-92.

- [18] Klein G, Phillips J K, Rall E L, et al. A Data-Frame Theory of Sensemaking[M] //Expertise Out of Context. 2007: 113-155.
- [19] Pirolli P, Card S K. The Sensemaking Process and Leverage Points for Analyst Technology as Identified Through Cognitive Task Analysis[C]//Proceedings of the 2005 International Conference on Intelligence Analysis. 2005.
- [20] Kadivar N, Chen V, Dunsmuir D, et al. Capturing and Supporting the Analysis Process[C]//Proceedings of the IEEE Symposium on Visual Analytics Science and Technology. 2009: 131-138.
- [21] Xu K, Attfield S, Jankun-Kelly T J, et al. Analytic Provenance for Sensemaking: A Research Agenda[J]. IEEE Computer Graphics and Applications, 2015, 35(3): 56-64.
- [22] Starr A, Storey M-A. Theory of Troubleshooting: The Developer's Cognitive Experience of Overcoming Confusion[J/OL]. ACM Transactions on Software Engineering and Methodology, 2026. DOI: 10.1145/3800945.
- [23] Miller T. Explanation in Artificial Intelligence: Insights from the Social Sciences [J/OL]. Artificial Intelligence, 2019, 267: 1-38. DOI: 10.1016/j.artint.2018.07.007.

- [24] Abdul A, Vermeulen J, Wang D, et al. Trends and Trajectories for Explainable, Accountable and Intelligible Systems: An HCI Research Agenda[C/OL]// Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems. 2018: 582:1-582:18. DOI: 10.1145/3173574.3174156.
- [25] Kaur H, Nori H, Jenkins S, et al. Interpreting Interpretability: Understanding Data Scientists' Use of Interpretability Tools for Machine Learning[C/OL]// Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems. 2020: 1-14. DOI: 10.1145/3313831.3376219.
- [26] Sieker J, Kramer M, Escoffier T, et al. The Illusion of Competence: Evaluating the Effect of Explanations on Users' Mental Models of Visual Question Answering Systems[C/OL]// Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024: 19459-19475. DOI: 10.18653/v1/2024.emnlp-main.1084.
- [27] BUÇINCA Z, Malaya M B, Gajos K Z. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-Assisted Decision-Making[J/OL]. Proceedings of the ACM on Human-Computer Interaction, 2021, 5(CSCW1): 188:1-188:21. DOI: 10.1145/3449287.

- [28] Pareek S, Govers J, Als N K K, et al. Sensemaking in Multi-Agent LLM Interfaces: How Users Interpret Transitions, Roles, and Reliability[C/OL]//Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026: 1-20. DOI: 10.1145/3772318.3791157.
- [29] Kumar A, Bajpai Y, Gulwani S, et al. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild[C/OL]//Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering. 2025: 432-444. DOI: 10.1109/ASE63991.2025.00043.
- [30] Vicente K J, Rasmussen J. Ecological Interface Design: Theoretical Foundations [J/OL]. IEEE Transactions on Systems, Man, and Cybernetics, 1992, 22(4): 589-606. DOI: 10.1109/21.156574.
- [31] Fonseca R, Porter G, Katz R H, et al. X-Trace: A Pervasive Network Tracing Framework[C]//Proceedings of the 4th USENIX Symposium on Networked Systems Design and Implementation. 2007: 271-284.
- [32] Sigelman B H, Barroso L A, Burrows M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure: dapper-2010-1[R]. Google, 2010.
- [33] Mace J, Roelke R, Fonseca R. Pivot Tracing: Dynamic Causal Monitoring for

- Distributed Systems[C]//Proceedings of the 25th ACM Symposium on Operating Systems Principles. 2015: 378-393.
- [34] Ragan E D, Endert A, Sanyal J, et al. Characterizing Provenance in Visualization and Data Analysis: An Organizational Framework of Provenance Types and Purposes[J]. IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 31-40.
- [35] Sacha D, Senaratne H, Kwon B C, et al. The Role of Uncertainty, Awareness, and Trust in Visual Analytics[J]. IEEE Transactions on Visualization and Computer Graphics, 2016, 22(1): 240-249.
- [36] Hullman J. Why Authors Don't Visualize Uncertainty[J]. IEEE Transactions on Visualization and Computer Graphics, 2020, 26(1): 130-139.
- [37] Dritsa D, Houben S. How Design Researchers Make Sense of Data Visualizations in Data-Informed Design Processes: The Role of Uncertainty[J/OL]. ACM Transactions on Computer-Human Interaction, 2024, 31(6): 72:1-72:53. DOI: 10.1145/3685268.
- [38] Guo H, Huang J, Laidlaw D H. Representing Uncertainty in Graph Edges: An Evaluation of Paired Visual Variables[J/OL]. IEEE Transactions on Visualization

- and Computer Graphics, 2015, 21(10): 1173-1186. DOI: 10.1109/TVCG.2015.2424872.
- [39] Ehsan U, Liao Q V, Passi S, et al. Seamful XAI: Operationalizing Seamful Design in Explainable AI[J/OL]. Proceedings of the ACM on Human-Computer Interaction, 2024, 8(CSCW1): 119:1-119:29. DOI: 10.1145/3637396.
- [40] AlSayyad A, Huang K Y, Pal R. AgentTrace: A Structured Logging Framework for Agent System Observability[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2602.10133>. arXiv: 2602.10133.
- [41] Zheng Y, Hu Y, Yu T, et al. AgentSight: System-Level Observability for AI Agents Using eBPF[PP/OL]. arxiv(2025) [2026-09-20]. <https://arxiv.org/abs/2508.02736>. arXiv: 2508.02736.
- [42] Lu J, Pan B, Chen J, et al. AgentLens: Visual Analysis for Agent Behaviors in LLM-Based Autonomous Systems[J]. IEEE Transactions on Visualization and Computer Graphics, 2025, 31(8): 4182-4197.
- [43] Zhao X, Sohn J, Zheng Q, et al. AgentGUI: An Interface for Observing and Steering Long-Running AI Agents[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2607.26300>. arXiv: 2607.26300.

- [44] Wu Z, Cho S, Villalobos C E M, et al. AgentGraph: Trace-to-Graph Platform for Interactive Analysis and Robustness Testing in Agentic AI Systems[C]//Proceedings of the AAAI Conference on Artificial Intelligence: vol. 40: 48. 2026: 41721-41723.
- [45] Gao T, Li H, Li J H, et al. Graph of Trace: Visualizing Execution Traces of Scientific Agents[C]//Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics: System Demonstrations. 2026: 297-306.
- [46] Epperson W, Bansal G, Dibia V, et al. Interactive Debugging and Steering of Multi-Agent AI Systems[C]//Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems. 2025: 156:1-156:15.
- [47] Hutter R, Pradel M. AgentStepper: Interactive Debugging of Software Development Agents[PP/OL]. arxiv(2026) [2026-09-20]. <https://arxiv.org/abs/2602.06593>. 3. arXiv: 2602.06593.
- [48] Dibia V, Chen J, Bansal G, et al. AutoGen Studio: A No-Code Developer Tool for Building and Debugging Multi-Agent Systems[C]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 2024: 72-79.

- [49] Rorseth J, Godfrey P, Golab L, et al. LADYBUG: an LLM Agent DeBUGger for Data-Driven Applications[C]//Proceedings of the 28th International Conference on Extending Database Technology. 2025: 1082-1085.
- [50] Parnin C, Orso A. Are Automated Debugging Techniques Actually Helping Programmers?[C]//Proceedings of the 20th International Symposium on Software Testing and Analysis. 2011: 199-209.
- [51] 范俊君, 田丰, 杜一, 等. 智能时代人机交互的一些思考[J/OL]. 中国科学: 信息科学, 2018, 48(4): 361-375. DOI: 10.1360/N112017-00221.
- [52] 范向民, 范俊君, 田丰, 等. 人机交互与人工智能: 从交替浮沉到协同共进[J/OL]. 中国科学: 信息科学, 2019, 49(3): 361-368. DOI: 10.1360/N112018-00181.
- [53] 孔祥维, 唐鑫泽, 王子明. 人工智能决策可解释性的研究综述[J/OL]. 系统工程理论与实践, 2021, 41(2): 524-536. DOI: 10.12011/SETP2020-1536.
- [54] 李瑶, 左兴权, 王春露, 等. 人工智能可解释性评估研究综述[J/OL]. 导航定位与授时, 2022, 9(6): 13-24. DOI: 10.19306/j.cnki.2095-8110.2022.06.003.
- [55] 邢颖. 基于可解释性人工智能的软件工程技术方法综述[J/OL]. 计算机科学, 2023, 50(5): 3-11. DOI: 10.11896/jsjcx.221100159.

华中科技大学

研究生（文献阅读与选题）报告

☐文献阅读、☐选题、☐文献阅读与选题、☐论文中期进展

题目：大语言模型智能体故障诊断的证据链交互设计研究

学号 M202570493

姓名 陈思静

专业 设计学

指导教师 何心

院（系、所） 设计学院